

ArchEval: Measuring AI Agents as Computer Architects

Chenyu Wang^{*1}, Zishen Wan^{*1}, Jeffrey Ma¹, Shvetank Prakash¹, Zhenting Qi¹, Haebin Do¹, Andy Cheng¹, Arya Tschand¹, Jiahe Shi², Yilun Du¹, Vijay Janapa Reddi¹

¹Harvard University ²Massachusetts Institute of Technology
USA

Abstract

Computer architecture has long used benchmarks to make progress measurable. LLM agents create a different measurement problem: when asked to act as computer architects, success is not merely writing code or tuning parameters. The agent must interpret workloads, choose mechanisms, use simulators, predict performance before feedback, satisfy hard constraints, and decide which feasible design is worth evaluating.

This paper introduces ArchEval, a benchmark and platform for evaluating LLM agents on computer architecture design and optimization. It contains 20 challenges across CPU core mechanisms, system architecture, memory systems, accelerators, and compute-in-memory, backed by eight simulators. Each challenge is posed under three evaluation settings: L1 *full harness*, with a prepared harness and repeated simulator feedback; L2 *simulator-code container*, where simulator source is available but the agent must assemble its own experimental workflow; and L3 *agent-only*, where the agent receives static workload evidence and constraints but no runnable simulator feedback before final submission. Each run reports baseline-normalized verifier performance and records the full trajectory, connecting final results to workload analysis, simulator-tool use, prediction, constraint handling, and artifact integrity.

Initial results show a sharp boundary in current agents. With L1 support, all four evaluated agents reach or exceed the challenge baseline and improve real architecture designs across diverse simulators. Removing support exposes diagnosable weaknesses: many agents do not turn simulator source into useful local experiments, and their L3 performance predictions often disagree with verifier results. In L3, only GPT-5.5 + Codex remains above baseline, reaching $1.21\times$ geomean baseline-normalized performance and a 65% win rate; the other three agents fall below baseline. Even GPT-5.5 + Codex has only a 15% performance-modeling pass rate. ArchEval therefore frames today’s agents as useful optimization assistants rather than autonomous architects, and identifies the capabilities needed next: simulator-tool use, calibrated performance prediction, pre-feedback design judgment, and useful mechanism discovery.

1 Introduction

For decades, computer architecture has marked progress by building benchmarks for the systems it learned to design: SPEC for single processors, PARSEC and CloudSuite for workloads, and MLPerf for machine-learning hardware [2, 6, 10, 40]. These suites evaluate artifacts and workloads. They do not evaluate the designer who chose the objective, mechanism, simulator, and constraints. For human designers, that separation was natural: judging the designer was more a matter of education and hiring than an architecture-benchmark problem. That boundary is beginning to move. LLM agents now fix

^{*}Equal contribution.

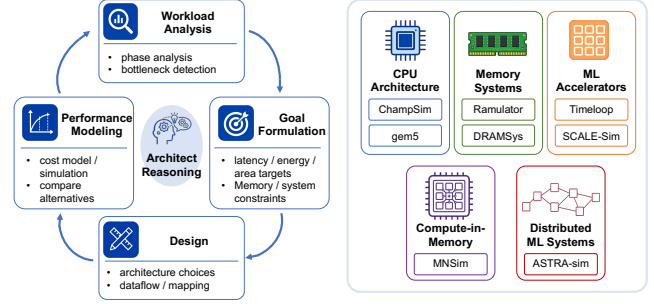


Figure 1: ArchEval evaluates LLM agents as computer architects. (a) Existing architecture benchmarks primarily measure completed designs, not the design process. (b) ArchEval asks whether LLM agents can perform the architecture reasoning process behind them: analyzing workloads, proposing mechanisms, modeling performance, and iterating across simulator-backed design domains.

real software bugs [49], reason through hard problems [28], solve open problems in mathematics [32], and have begun to take on computer architecture design itself [4, 12, 45, 47]. Recent systems work frames this software-to-silicon shift as a recurring challenge around tool interfaces, verification, and feedback loops [44]. Computer architecture now faces a new question: *can an LLM agent do the work of a computer architect, and how would we measure it?*

Consider how a computer architect actually works. Architecture design is a loop (Figure 1): analyze the workload, choose or invent a mechanism, model how the result will perform, and refine. Within this broader loop, architects also run optimization loops, iterating over candidate configurations once the objective, interface, metric, and evaluator are fixed. The classic example is design space exploration (DSE) [22, 24, 52], and machine learning already automates parts of that loop, from search to learned cost and workload models [1, 14, 30]. The design loop is broader. It has largely stayed human, and it is where many architectural shifts begin: reframings like quantization, sparsity, and disaggregated serving [13, 35, 36] would not naturally emerge from search over a narrowly predefined space. Evaluating an agent as an architect therefore means testing both feedback-driven optimization and design-loop judgment.

Building such a benchmark is hard for four reasons. First, there is no universal way to compare design performance across simulators. A mathematical solution can be checked against a known answer and a program can simply be run, but a computer-architecture design must be measured by the simulator built for that domain or by taping out a chip. Second, that measurement is slow: a single simulation can take hours, days, or months, and a tape-out takes far longer. Third, expertise is scattered across specialties, from distributed systems and memory to CPU cores, ML accelerators, GPUs,

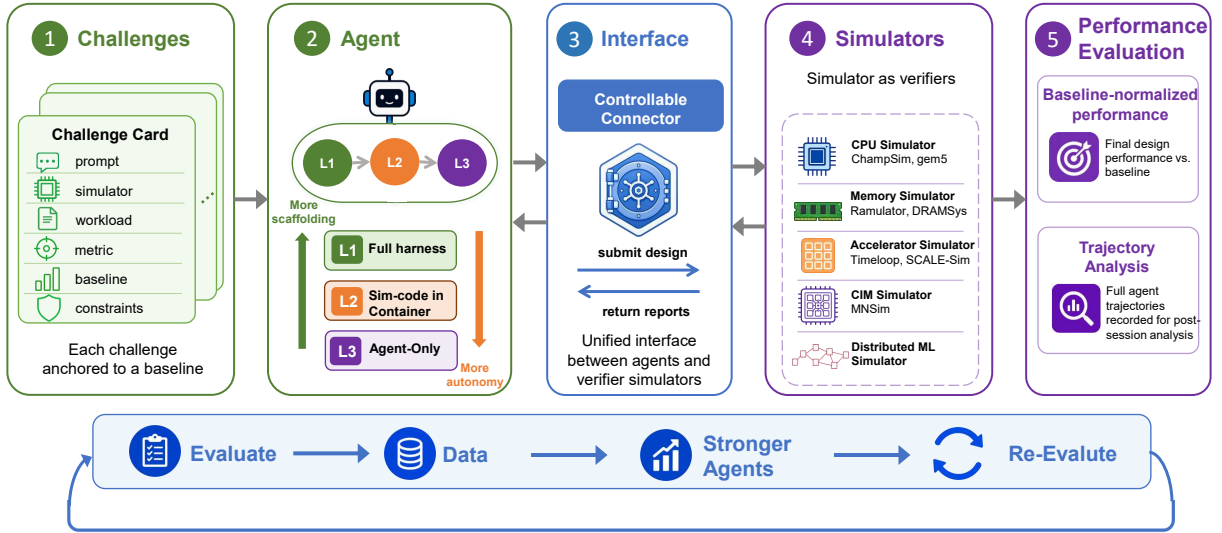


Figure 2: ArchEval evaluates agents through a controlled architecture-design workflow. Each challenge provides a task, baseline, and evaluation setting; agents produce design artifacts; a connector mediates simulator and verifier access; and ArchEval measures final designs while recording trajectories for process analysis. The resulting data can guide the next generation of architecture agents.

and Compute-in-Memory (CIM). Fourth, agent autonomy is ambiguous. A strong final result can mean that the agent designed well, but it can also mean that humans supplied the design space, simulator workflow, and feedback loop. The benchmark must therefore measure both the final artifact and the support the agent needed to produce it.

In this work, we build ArchEval, a systematic benchmark to evaluate LLM agents on computer architecture design and optimization. To make comparison possible, we put diverse simulators behind a single interface and measure every submitted design against a credible per-challenge baseline. To make evaluation practical, we use lightweight but simulator-backed challenges with bounded verifier runtimes. To reflect the breadth of the field, the suite spans CPU cores, memory systems, accelerators, distributed training, and CIM.

ArchEval’s defining feature is that it also varies the experimental support available to the agent. On every challenge, the agent runs in one of three settings (Figure 2). L1 full harness gives the agent a prepared optimization loop for design-space exploration. L2 simulator-code container gives the simulator source and build environment, but asks the agent to build and drive its own experimental workflow. L3 agent-only targets the stage before such a workflow is available: the agent receives static workload evidence and constraints, but no runnable simulator or simulator feedback, and must estimate performance before submission. These settings make the motivating question measurable: how much of an architect’s work can an agent already own, and which parts fail when the prepared harness and simulator feedback are removed?

Evaluating agents across these settings gives a capability map, not just a leaderboard. In L1, where a prepared harness supplies the experimental loop, agents have crossed an important threshold: they can already improve real architecture designs across diverse simulators. But the capability is uneven. When the prepared harness is removed, performance drops in diagnosable ways. L2 shows

whether an agent can use simulator source to build a working experimental workflow. L3 shows whether it can use static workload evidence to choose a feasible design and predict its performance before verifier-simulator feedback. In our final single-seed suite, all four evaluated agents reach or exceed the challenge baselines at L1, but only GPT-5.5 remains above baseline at L3; the other three agents fall below baseline once simulator feedback is removed. The trajectory records explain the drop: agents struggle to assemble evaluation workflows, build workload-grounded performance models, and decide which feasible design is worth trying. The failure is not simply artifact generation: many L3 submissions are runnable and valid. The missing capability is design judgment before feedback, where agents often cannot tell which feasible design is actually better. These failures are not merely negative results; they define the roadmap for the next generation of architecture agents.

This paper therefore makes three contributions:

- **A capability map for AI computer architects.** ArchEval frames architecture-agent evaluation as more than final design performance. It separates prediction before feedback, optimization with feedback, and generation of concrete architecture artifacts. Its L1/L2/L3 settings hold the task fixed while progressively removing the prepared harness and simulator feedback, exposing assisted DSE, simulator-tool use, and pre-feedback design judgment.
- **A controlled multi-simulator platform.** ArchEval unifies eight architecture simulators behind a common connector, evaluates each design against a credible per-challenge baseline, enforces hard constraints with an isolated verifier, and records full trajectories. This makes architecture-agent evaluation simulator-grounded, comparable within each challenge, and auditable beyond a single final result.
- **An empirical study of current agent configurations.** We evaluate four agent configurations on 20 challenges

Table 1: Architecture-agent capability map. ArchEval keeps the high-level roles of prediction, optimization, and generation, then maps each one to concrete architecture work, open capability questions, and the corresponding evaluation signal.

Capability	Architecture work	What agents can plausibly attempt today	What is not yet established	How ArchEval tests it
Prediction	Analyze workloads	Produce workload-analysis notes and inspect visible traces.	Whether the analysis identifies the true bottleneck and changes the submitted design.	Workload-to-design analysis.
	Model performance	Write an agent-authored performance model or prediction script.	Whether the model ranks candidate designs consistently with the verifier simulator and calibrates uncertainty.	Performance-prediction diagnostic.
Optimization	Search a fixed design space	Edit configurations or design files, use a prepared harness, and improve a measured metric.	Whether improvement reflects broader design judgment rather than harness-guided search.	Feedback-loop optimization analysis.
	Use simulator tools	Read source files, build tools, run scripts, and parse logs.	Whether the agent can assemble a reliable experimental workflow from simulator access.	Simulator-use trajectory analysis.
Generation	Choose design mechanisms	Recombine known policies, propose mechanisms, and tune parameters.	Whether the agent can connect a bottleneck to a mechanism rather than template-match a familiar design.	Design-novelty analysis.
	Handle constraints	Follow explicit, checkable limits such as storage, area, topology, accuracy, and output format.	Whether the agent can choose the best feasible design within those constraints.	Constraint-following and shortcut/interface-failure analyses.

across CPU, system, memory, accelerator, and CIM domains. The results reveal a sharp boundary: agents can improve real designs with the full harness, but often fail to build useful local experiments or judge feasible designs before simulator feedback. Trajectory analysis traces the drop to weak simulator-tool use, unreliable performance prediction, overestimated feasible designs, and limited mechanism discovery.

The remainder of this paper is organized as follows. Section 2 develops the capability framing. Section 3 defines the ArchEval benchmark protocol. Section 4 describes the platform that runs the protocol across heterogeneous simulators. Section 5 presents the empirical study by evaluation setting, and Section 6 summarizes the capability findings that cut across those settings. Section 7 positions ArchEval against related benchmarks and methods. Section 8 discusses implications, limitations, and community contributions, and Section 9 concludes.

2 Evaluating AI Agents as Architects

A benchmark for AI architects should measure what an architect must do, rather than treating one high-performing artifact as the whole answer. We organize that work around three roles that recur in learning-enabled systems design: *prediction*, *optimization*, and *generation*. Prediction estimates the behavior of a candidate before the expensive measurement is available. Optimization decides which candidate to try next when some feedback signal exists. Generation produces the architecture artifact itself. Modern agentic systems often combine all three, but separating them is useful because each role fails in a different way.

Computer architecture makes this separation especially important. The final answer is usually determined by a simulator, compiler, deployment stack, or silicon result, but those measurements are costly and incomplete. A useful agent must therefore predict when feedback is missing, optimize when feedback is available, and generate concrete mechanisms under real constraints. ArchEval uses this capability view to ask a more precise question than whether agents can “do architecture”: which parts of architectural work can they already own, and which parts still require human support?

2.1 Prediction: Modeling Architecture Behavior Before Measurement

Prediction is the ability to judge a candidate design before the canonical measurement is available. In computer architecture, that judgment may estimate performance, energy, area, accuracy, memory traffic, communication cost, or constraint feasibility. This is not a secondary skill. Architects routinely make early decisions before a full simulator, compiler flow, or deployment stack can give the final answer, let alone silicon measurement. They rely on workload evidence, analytical models, surrogates, and experience to decide whether an idea is worth implementing.

Learning has long served this role in systems by replacing expensive evaluation with cheaper cost models and performance predictors [1, 14, 23, 30]. The recurring risk is the proxy gap: a cheap predictor may correlate with the target metric on familiar cases while misleading design under a new workload, architecture, or constraint. For an AI architect, this gap becomes a central capability question. The agent must know when its own model is reliable, when uncertainty is high, and when a seemingly plausible design is likely to fail once the verifier simulator runs.

2.2 Optimization: Searching with Evaluation Feedback

Optimization is the ability to use feedback to improve a design. Once the objective, design interface, metric, and evaluator are fixed, the task becomes a search over candidate designs. Design space exploration (DSE) is the classic architecture example: a human defines the design space and simulator, then a method searches for a configuration that improves the metric [22, 24, 52]. This is a natural place for agents to help. A tool-using agent can propose a candidate, read a measurement, diagnose the failure, and try a different point.

Optimization is also where it is easiest to overstate autonomy. Strong DSE performance does not imply that the agent has framed the architecture problem. The design objective, legal choices, simulator path, and feedback loop may all have been supplied by humans. DSE is therefore an important first capability, but not the whole architect’s job. The harder question is whether the agent can build or adapt the evaluation loop when the ready-made harness is removed, and whether it can still make sound decisions when measurement feedback is unavailable.

2.3 Generation: Producing Architecture Artifacts and Mechanisms

Generation is the ability to produce the candidate architecture artifact: a policy, mapping, configuration, microarchitectural mechanism, or co-design choice that can be checked by architecture tools. Unlike ordinary code generation, architecture generation has a physical interpretation. A few lines of C++ may represent SRAM state and replacement logic; a YAML mapping may encode data reuse, buffer pressure, and spatial parallelism; a system configuration may determine communication distance and collective scheduling. When required, the agent may also generate auxiliary artifacts such as design reports or performance-model scripts, but those artifacts support prediction; the core generated object remains the architecture design. A generated artifact is useful only if it both satisfies the tool interface and embodies a design mechanism that is meaningful under the workload, constraints, and target metric.

This makes generation different from text completion. The agent must translate a high-level task into an artifact that compiles, runs, satisfies hard constraints, and corresponds to the design argument it gives. It must also decide whether it is merely recombining known mechanisms or proposing a substantive new one. Generation without prediction can produce plausible but weak designs. Generation without optimization can miss easy improvements. Generation without constraints can win the metric by leaving the architecture problem behind.

2.4 What an Architecture-Agent Benchmark Must Measure

A benchmark should make these roles observable without forcing them into a one-to-one mapping with protocol settings. Prediction, optimization, and generation are capability roles; evaluation settings are ways to vary the support available to the agent. Each architecture task can involve all three roles in different proportions. The benchmark must therefore ask what the agent predicted, how it used feedback when feedback existed, what artifact it generated, and whether the process that produced the artifact is visible. This is why ArchEval poses the same challenge under multiple settings: the architecture task stays fixed, while the available evidence, tools, and feedback change.

Table 1 summarizes this capability map. The left column gives the three roles, while the remaining columns make them operational as concrete architecture work. Final performance can tell whether the submitted design beat a baseline, but it cannot tell whether the agent predicted well, optimized through evidence, or generated a valid architecture mechanism for the right reason. For that reason, ArchEval records both final performance and the trajectory that produced it.

This framing gives four requirements for an architecture-agent benchmark. First, it must be *architecture-specific*: the agent should produce concrete architecture artifacts rather than answer questions about architecture concepts alone. Second, it must be *execution-grounded*: designs should be measured by simulators and baselines, because architecture quality is measured through behavior, not surface form. Third, it must *vary experimental support*, so the benchmark can distinguish an agent that searches inside a prepared loop from one that can assemble or replace that loop or reason before

feedback exists. Fourth, it must include *process evidence*, because final baseline-normalized performance alone cannot show whether the agent understood the workload, built a working evaluation loop, trusted a flawed predictor, found an interface shortcut, or guessed a good configuration. Section 3 turns these requirements into the ArchEval protocol: a challenge format, evaluation settings with different levels of support, baseline-normalized performance, and trajectory analysis.

3 The ArchEval Benchmark Design

ArchEval defines a simulator-grounded evaluation protocol for computer architecture agents. Each run is specified by a challenge, an evaluation setting, an agent, and a verifier. The challenge card in Table 2 defines the full task specification: architecture objective, workload evidence, deliverable interface, verifier, metric, baseline, hard constraints, and visibility and feedback policy. The setting determines what experimental support the agent can access before submission. The verifier runs the submitted artifact against the canonical simulator and reports validity, constraint status, native metrics, and baseline-normalized performance. In addition to the final performance, ArchEval records the full trajectory so that the submitted design can be interpreted together with the process that produced it. This section defines that protocol, and Section 4 describes the platform that implements it across simulators.

3.1 What is an ArchEval Challenge?

An ArchEval challenge is a simulator-grounded design problem. We represent each challenge as a *challenge card*: the stable specification shared by the benchmark and a run. Some fields are agent-visible, such as the prompt, deliverable interface, workload evidence, metric, baseline description, and constraints. Other fields, such as the canonical verifier script and hidden reference state, are benchmark-internal and are mediated by the visibility and feedback policy. The card states what the agent may see, what it must submit, what the verifier will measure, and which constraints are non-negotiable. The deliverable may be source code, a configuration, a mapping, a co-design artifact, or a written model, depending on the subfield. The verifier then runs that deliverable in the challenge’s simulator and reports a typed outcome.

Two examples make this structure concrete. First, in a cache replacement challenge, the task asks the agent to write an LLC replacement policy for ChampSim that beats a given baseline policy. The deliverable is a C++ policy file, the workload is a fixed trace set, the metric is IPC, and the baseline is the given baseline policy. Second, in a CIM co-design challenge, the task asks the agent to produce a network-and-accelerator design JSON for a ReRAM-CIM accelerator. The verifier runs MNSIM, checks an accuracy floor and area cap, and reports energy-delay product (EDP) relative to a Gibbon-based baseline [41]. The artifact and simulator differ, but the evaluation flow is the same: the agent receives a challenge card, works in its setting-specific workspace, submits an artifact to a canonical verifier, and is measured against the challenge baseline. A run proceeds in four steps: the agent receives the setting-specific challenge package, edits or creates the required deliverable in its workspace, submits the artifact to the verifier, and receives either a typed failure or a valid metric report.

Table 2: Challenge card schema. Each ArchEval challenge exposes the same protocol fields even when the underlying simulator and artifact type differ.

Field	Meaning
Prompt	architecture objective and task context
Deliverable	file, configuration, mapping, model, or co-design artifact to submit
Workload evidence	traces, summaries, model layers, network description, or other task evidence
Verifier	canonical simulator and evaluation script used by the benchmark; visibility depends on setting
Metric	simulator-native metric and direction
Baseline	reference design used for normalization
Constraints	hard validity, resource, accuracy, topology, or format rules
Visibility and feedback	setting-specific source access, local tool access, and verifier-feedback policy

3.2 Challenge Suite and Baselines

ArchEval includes 20 challenges that span CPU core mechanisms, system architecture, memory systems, accelerators, and CIM. Each challenge is anchored to a credible baseline: either a standard design from practice, such as LRU for cache replacement, or a published design, such as Gibbon [41] for CIM co-design.

The baseline is not an oracle optimum. It is a credible reference point that makes results comparable within a challenge. Some baselines are standard practice, some are simulator defaults or hand-written references, and some come from published designs. We report relative performance together with win rate and hard-failure counts so that beating a baseline is not treated as proof of global optimality. The baseline and submitted design are evaluated by the same verifier, workload, hardware configuration, metric parser, and constraint checker.

We include many challenges because architecture design is not one task repeated under different names. A challenge enters the suite only if it asks for an architecture-level design artifact, has a simulator-backed verifier, defines a baseline and simulator-native metric, and can run within the evaluation budget. One design choice is worth making explicit: the individual tests are intentionally lightweight. Real architecture evaluation can take days, weeks, or months, but that timescale is a poor fit for an agent benchmark: by the time a long simulation campaign finishes, the underlying LLM may already have changed. We therefore choose tasks whose verifiers are fast enough to support repeated agent evaluation, while still requiring concrete architecture artifacts and simulator-backed measurement. Lightweight does not mean toy: each challenge isolates one architecture design decision with a real simulator, workload, baseline, and hard constraints. New challenges can be added by supplying the same pieces: task prompt, deliverable format, workload, baseline, metric parser, constraint checker, and verifier script. Table 3 summarizes the suite by design domain; Appendix A gives the full 20-challenge manifest with deliverables and constraints.

3.3 Evaluation Settings: L1 to L3

The defining feature of ArchEval is that the same challenge can be posed at three evaluation settings that give the agent different

amounts of experimental support. These settings vary experimental support rather than capability type: each setting can exercise prediction, optimization, and generation, but changes which parts of that work are supplied by the benchmark and which parts the agent must own. Table 4 summarizes the support removed at each setting, and Figure 3 shows the same cache replacement challenge instantiated across L1 to L3.

L1 full harness gives the agent a prepared workflow: starter files, a narrow design interface, the complete simulator and evaluation toolchain, and repeated feedback from the verifier simulator. The agent’s action space is intentionally constrained; it mostly edits one or a few files and iterates through the provided harness. A strong L1 result corresponds to assisted design-space exploration: the agent can improve an architecture when humans have already built the objective, simulator path, and feedback loop.

L2 simulator-code container is the middle setting. The agent receives the simulator source, build environment, and task specification, but not a prepared optimization script. It must inspect the simulator, discover the relevant interfaces, write scripts or drivers, run local probes, parse outputs, and decide what feedback is trustworthy before submitting. This setting models the bridge between optimization and architecture engineering: the simulator exists, but the agent must turn it into actionable experimental feedback.

L3 agent-only removes runnable simulator tools from the agent’s workspace. The agent still receives a static task package: the design prompt, deliverable interface or starter files, hardware constraints, and workload evidence such as trace summaries or samples. It does not receive the simulator source, a buildable simulator workflow, local simulator runs, or iterative verifier feedback. It must design the artifact and construct a performance model before making a final verifier submission. This setting targets early architecture design, where a usable simulator often does not yet exist and the architect must translate the objective and workload evidence into a proxy model before deciding whether an idea is worth implementation. L3 tests performance modeling before simulator feedback rather than search inside a loop that humans have already built.

different meanings by setting. In L1, repeated valid verifier measurements are intended feedback for optimization. In L2 and L3, failed build, validation, or timeout attempts can be retried up to the cap, but the first valid verifier result terminates the session. Thus valid verifier measurements in L2 and L3 cannot become a design-space-exploration loop.

Table 5 summarizes the capability coverage induced by this protocol. It shows how the protocol exposes the capability dimensions defined in Table 1 and how that coverage differs from representative benchmarks and LLM-for-architecture methods. The lower rows show how L1, L2, and L3 expose increasing parts of the architecture-agent capability map. The architect-ability columns are a compact projection of Table 1: workload analysis and performance prediction instantiate prediction, design-space exploration and evaluation-loop assembly instantiate optimization, and constraint-aware trade-offs instantiate constrained generation.

3.4 Scoring, Validity, and Aggregation

Every valid final design is measured against its challenge baseline. We report the result as baseline-normalized performance,

Table 3: ArchEval spans the main architecture design domains. The suite has 20 challenges across five domains and eight verifier simulators. Baseline-normalized performance values above 1.0× beat the baseline.

Domain	Simulator	N	Varies	Purpose
CPU core mechanisms	ChampSim [11]	6	branch prediction, BTB design, cache replacement, prefetching, and composition	tests classic microarchitecture choices with tight implementation interfaces
System architecture	gem5 [3], ASTRA-sim [48]	6	cache hierarchy configuration and collective-communication choices	tests system-level design across memory hierarchy and distributed training workflows
Memory systems	DRAMSys [19], Ramulator [21]	2	controller tuning and RowHammer mitigation	tests timing-sensitive policies and hard correctness or protection constraints
DNN accelerators	SCALE-Sim [39], Timeloop [34]	4	array size, workload partitioning, and dataflow mapping	tests accelerator scheduling, mapping, and energy-performance tradeoffs
CIM and co-design	MNSim [54]	2	CIM design-space search and neural-network plus hardware co-design	tests cross-layer design under accuracy, area, and energy constraints

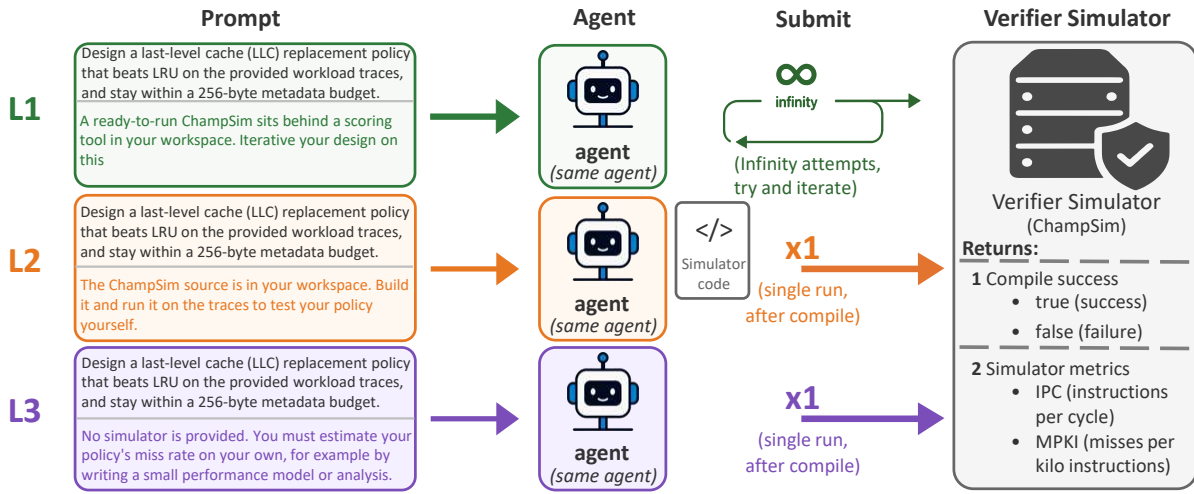


Figure 3: One task, three evaluation settings. A worked example: the same cache replacement task is posed under L1 to L3 using the same agent. What changes is the prompt’s requirement, the experimental support available to the agent (L2 gets simulator source; L3 gets only static workload evidence), and how verifier feedback is used. L1 can iterate on repeated measurements; L2 and L3 stop at the first valid verifier result, although failed attempts may be retried up to the challenge cap. All three submit to one shared canonical verifier (ChampSim), which reports whether the design compiles and its simulator metrics (IPC, MPKI).

Table 4: The three settings remove experimental support step by step. Within a challenge, the task, workload, verifier simulator, and baseline stay fixed.

Setting	Provided support	What it tests
L1 full harness	prepared harness, narrow design interface, and repeated verifier-simulator feedback	search with a prepared harness and repeated feedback
L2 simulator-code container	simulator source, build system, and local probes or runs when available; final verifier stops at first valid result	assembling simulator tools into actionable experimental feedback
L3 agent-only	static task package; no simulator source, runnable simulator, or local simulator feedback	performance prediction before simulator feedback

metrics where higher is better, such as IPC or bandwidth, we compute $m_{\text{agent}}/m_{\text{base}}$. For metrics where lower is better, such as cycles, MPKI, overhead, or energy-delay product, we compute $m_{\text{base}}/m_{\text{agent}}$.

Each verifier run also returns a typed outcome, such as SIM_OK, BUILD_FAIL, VALIDATION_REJECT, or SIM_TIMEOUT, plus the simulator-native metric blob when successful. A design is a hard failure if it does not build, times out, misses a required output, violates a hard constraint, or cannot be evaluated by the canonical simulator. We keep these failures visible because they are part of agent capability: an agent must produce a valid design, not just a promising idea.

Aggregates are computed over valid baseline-normalized performance values unless a table or figure states otherwise. To avoid hiding invalid submissions, we pair valid-only geomean and median with win rate over all challenges and explicit hard-failure counts. Invalid submissions count as non-wins in win-rate calculations.

with values above 1.0× meaning the agent beats the baseline. For

Table 5: Protocol coverage and positioning. The left columns compare benchmark scope: public tasks, agent setting, architecture-specific tasks, verifier simulators, and domain coverage. The right columns summarize the architecture-agent abilities exposed by the protocol, using a compact projection of the capability map in Table 1. ArchEval tasks are intended for release with the benchmark.

	Benchmark scope					Architect ability				
	Public tasks	Agent setting	Architecture tasks	Verifier simulator	Multiple domains	Workload analysis	Design-space exploration	Build eval loop	Constraint-aware trade-off	Predict performance
<i>Benchmarks</i>										
SWE-bench [18]	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
Terminal-bench [31]	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
QuArch [37]	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗
<i>LLM-for-architecture methods</i>										
LLM-driven DSE [45, 47]	✗	✓	✓	✓	✗	✗	✓	✗	✗	✗
ArchAgent [12]	✗	✓	✓	✓	✗	✗	✓	✗	✗	✗
Agentic Architect [4]	✗	✓	✓	✓	✗	✗	✓	✗	✗	✗
<i>Ours</i>										
ArchEval (L1)	✓	✓	✓	✓	✓	✗	✓	✗	✓	✗
ArchEval (L2)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗
ArchEval (L3)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

A hard-constraint violation removes the design from valid performance aggregation, while the trajectory rubric can still record the violation as evidence about constraint awareness. Section 4 describes how the connector implements these outcomes across simulators.

3.5 Post-Session Trajectory Evaluation

Final performance alone is insufficient for ArchEval because two trajectories can produce the same final performance while exercising different capabilities. One agent may analyze the workload, write a design document, build a local experiment loop or agent-written performance model, and revise the design using evidence. Another may skip the intermediate work and submit only a final artifact. The final performance can be identical, but the capability demonstrated by the two runs is different.

For this reason, ArchEval records the full trajectory of every session: the prompt, visible rationale when available, tool calls, simulator observations, submitted artifacts, file diffs, build logs, verifier outcomes, and the timing of each step. We analyze these records with the rubric in Table 6. The rubric complements final performance by checking process dimensions that are visible in the logs. Task compliance is a cross-cutting sanity check; workload-grounded design and performance judgment expose prediction; simulator and tool use exposes optimization; and constraint awareness, integrity, and originality expose generation quality.

4 The ArchEval Platform

The benchmark design in Section 3 defines what an evaluation means. The ArchEval platform runs that workflow across heterogeneous architecture simulators. The platform has two goals: make different architecture simulators look uniform to the agent, and make every reported result reproducible and auditable. Its job is to give every challenge the same agent-facing workflow while keeping the canonical verifier isolated, the visibility policy explicit, and the full trajectory traceable. This section focuses on the engineering interface behind that platform: how we make a multi-simulator

Table 6: Post-session trajectory rubric. The rubric connects visible process evidence to the capability map in Table 1.

Dimension	What we check
Task compliance	Did the agent deliver the requested final artifact and intermediate artifacts, such as design notes or a performance model, in the required format?
Simulator and tool use	When tools are available, did the agent build, run, debug, and parse local experiments rather than treating simulator code as static reference text?
Workload-grounded design	Did the agent inspect the actual workload and use measured workload properties to choose or change the submitted design, rather than writing generic architecture intuition?
Performance judgment	When requested or when simulator feedback is unavailable, did the agent produce a runnable, design-sensitive performance model or self-evaluation that can distinguish good and bad candidate designs?
Constraint awareness	Did the agent keep the design within the required resource and validity constraints, such as storage budgets, area caps, topology limits, or accuracy floors, instead of improving the metric by using invalid resources?
Integrity and originality	Did the submitted artifact match the claimed design rationale, avoid starter-copying or interface shortcuts, and propose a substantive design rather than only trivial parameter changes?

benchmark uniform enough for agents to use, but controlled enough for baseline-normalized performance and trajectories to be trusted.

4.1 System Overview

Each run is organized as a handoff among four components. The *orchestrator* starts and bounds the run: it chooses the challenge and evaluation setting, creates the workspace, launches the agent, and stops the session when the budget is exhausted or the agent is done. The *connector* is the agent-facing interface. It exposes the same small set of actions for every challenge, such as submitting a design, checking a result, or reading allowed simulator files. The *configuration checker* verifies that the run uses the intended environment: simulator image, workload set, baseline, hardware configuration,

Table 7: The connector gives every simulator the same agent-facing tools. Source access is restricted by each setting and challenge blacklist.

Tool	Behavior
submit	queue a submission asynchronously; returns an id; a worker runs the challenge’s canonical <code>evaluate.sh</code> in an isolated simulator container
submit_and_wait	synchronous wrapper: submit, then poll to a terminal outcome
check_submission	poll: returns the typed outcome (SIM_OK / BUILD_FAIL / VALIDATION_REJECT / SIM_TIMEOUT) plus the metric blob on success
session_end	declare the session finished; triggers the grace-period drain of in-flight submissions before teardown
browse_simulator	list files in the simulator source tree, subject to the challenge’s source blacklist
read_simulator_file	read one simulator source file, anonymized and blacklist-checked

constraints, budgets, and source-visibility policy. This component exists because agent evaluation has many moving parts; without a separate consistency check, two runs can appear comparable while differing in small but important configuration details. The *monitor* persists the trajectory evidence produced during the run.

In Section 3, we use *verifier* for the canonical scoring role. In the platform, the connector does not score designs itself; it dispatches submissions to an isolated verifier-simulator container, then parses the native output into a typed outcome and baseline-normalized score. For example, in a ChampSim cache-replacement run, the orchestrator creates the setting-specific workspace; the connector receives a submitted policy file; the configuration checker verifies the workload set, baseline policy, simulator image, and storage budget; the isolated verifier container runs the canonical evaluation script; and the monitor persists the file diff, build log, typed outcome, IPC/MPKI metrics, and normalized score.

This division keeps the agent’s experience simple even though the back end is heterogeneous. Operationally, the protocol fields in Section 3 become the adapter artifacts described in Section 4.3. The connector uses the script and parser to turn the simulator’s native output into the baseline-normalized performance of Section 3.4; the configuration checker confirms that the run uses the intended configuration. The agent does not need to know whether the back end is ChampSim, gem5, Ramulator, Timeloop, or MNSim. It sees the same interaction pattern, while the platform keeps the verifier-simulator environment isolated and the full trajectory traceable.

4.2 Connector and Tool Interface

The connector is the mechanism that makes a multi-simulator benchmark usable by agents. It has to solve three problems at once. First, it normalizes submission: the agent always submits a candidate design and receives a typed evaluation result, even though the back end may be ChampSim, gem5, Ramulator, Timeloop, or MNSim. Second, it controls visibility. The agent must see enough of the simulator interface to do honest design work, but not the hidden references or answer artifacts that would collapse the benchmark [7, 12, 33]. Third, it emits structured events, so submissions, blocked source reads, build failures, validation rejections, timeouts, and valid metric reports can be persisted by the monitor as trajectory evidence.

Table 8: Tool availability follows the evaluation setting. Official scoring always goes through the isolated verifier; local runs are available only when the setting provides an executable workflow.

Support	L1	L2	L3
Submit, check, and end session	✓	✓	✓
Editable workspace	✓	✓	✓
Local harness or probes	✓	✓	✗
Simulator implementation source	interface/ starter only	✓	✗
Connector-mediated source browsing	challenge dependent	not needed	✗
Repeated valid verifier feedback	✓	✗	✗

A submission follows the same outcome lifecycle in every challenge. The connector first applies pre-simulation gates, such as format checks and statically checkable constraints, then build gates, then simulator execution. Format errors or pre-simulation constraint violations return `VALIDATION_REJECT`; compilation or linkage failures return `BUILD_FAIL`; executions that exceed wall-clock or resource limits return `SIM_TIMEOUT`. Constraints that require simulator output are checked after execution before the run is marked `SIM_OK`; successful verifier runs return `SIM_OK` with native metrics, constraint status, and a baseline-normalized score. Local experiments are advisory: only the isolated canonical verifier produces official typed outcomes and baseline-normalized scores.

All interactions with the canonical verifier therefore flow through the same tool interface (Table 7). The submission tools queue a candidate design and return a typed evaluation result. The lifecycle tools let the agent end a session cleanly. The source-browsing tools expose only selected simulator files, such as headers that define an interface, and pass every output through the connector’s blacklist and anonymizer. Table 8 summarizes how these tools and local supports differ by evaluation setting.

In L1, the workspace exposes the design interface and starter files; any additional simulator source is exposed only through challenge-controlled connector browsing. The key distinction between L1 and L2 is that L1 gives repeated verifier feedback through a narrow interface, while L2 gives broader simulator source access but no repeated valid verifier measurements.

4.3 Simulator Integration

ArchEval currently integrates simulators across CPU core mechanisms, system architecture, memory systems, accelerators, and CIM (Table 9). The platform treats each simulator as a canonical evaluation backend behind the connector. Each simulator adapter exposes the same interface: consume a submitted artifact, run the canonical evaluation script in an isolated container, and emit a typed outcome, native metric blob, constraint status, and baseline-normalized score. Internally, the adapter may be simulator-specific; at the platform boundary, it must expose a workspace template, `evaluate.sh`, metric parser, constraint checker, environment manifest, and visibility or blacklist policy. The parser maps native simulator metrics to the baseline-normalization rule of Section 3.4, while the configuration checker confirms that the expected build, runtime, constraint, and configuration gates are in place. Whether cached or re-evaluated, the baseline and submitted design use the same adapter, parser,

Table 9: The verifier simulators cover CPU, system, memory, accelerator, and CIM tasks. Each simulator follows the same connector protocol.

Simulator	Subfield	What it models
ChampSim [11]	CPU	core microarchitecture, caches, branch prediction
gem5 [3]	System	full-system cores, caches, and memory
Ramulator [21]	Memory	DRAM main memory
DRAMSys [19]	Memory	DRAM subsystem timing and power
ASTRA-sim [48]	System	training across compute and network
SCALE-Sim [39]	Accelerators	systolic-array DNN accelerators
Timeloop [34]	Accelerators	DNN accelerator dataflow and energy
MNSim [54]	CIM	memristor (ReRAM) neural accelerators

and constraint policy. The platform does not make native metrics comparable across simulators directly; comparability comes from per-challenge baseline normalization.

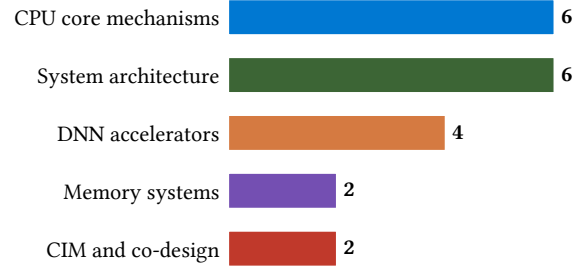
Evaluation also has to scale, so we target bounded verifier run-times, usually within a two-hour worst-case budget per canonical verifier attempt. This constraint shapes both challenge selection and simulator integration: a challenge can enter the suite only if its evaluation backend can produce a useful signal within the evaluation budget, even when agents submit multiple attempts or fail before producing a valid result.

4.4 Isolation, Visibility, and Logging

The platform separates the agent workspace from the verifier simulator. The agent can edit and run code in its own workspace, while the canonical verifier simulator, reference workloads, baselines, and simulator state live outside the writable environment. The connector mediates the verifier-simulator boundary: it enforces the source-visibility policy for each evaluation setting, strips or blocks disallowed files, and anonymizes exposed source when needed. When source must be exposed, anonymization removes metadata that would reveal hidden references or answer artifacts while preserving the interface information needed for implementation.

This isolation prevents several direct shortcuts: reading hidden references or answer artifacts, editing the canonical verifier, changing the workload set, metric parser, baseline, or constraints used for official scoring, and using a modified local simulator state as the official score. It does not prevent every interface shortcut. Agents can still overfit visible feedback, exploit a metric mismatch between the intended design objective and the exposed scoring interface, copy starter artifacts, or produce a submitted implementation that does not match the claimed rationale. Those behaviors are detected from trajectory records rather than prevented solely by isolation.

The monitor records the complete session: prompts, visible rationale when available, connector events, tool calls, observations, submitted artifacts, file diffs, build logs, evaluation outputs, timing, and terminal outcomes. Logs are stored with the challenge id, setting, agent configuration, container image, parser version, visibility policy, and terminal outcome, so a reported score can be traced back to the run that produced it. These logs do more than support reproducibility. They are the data used by the trajectory rubric in Section 3.5, which lets ArchEval explain why performance

**Figure 4: Challenge-suite composition.** Distribution of the 20 final challenges by architecture domain.

changes across evaluation settings and distinguish performance failures from simulator-tool, constraint, and shortcut failures.

5 Experiments

In this section we discuss early evaluation results from ArchEval. We first describe the experimental setup (Section 5.1), then report the main results across the three evaluation settings. The section follows the protocol introduced in Section 3.3: L1 tests optimization inside a prepared harness, L2 tests whether simulator source can become an experimental workflow, and L3 tests pre-feedback design judgment. The cross-cutting capability findings are separated into Section 6.

5.1 Evaluation Setup

We build the ArchEval platform and run all of our experiments on it. The challenges we test span the field: 20 challenges across eight simulators, covering CPU core mechanisms, system architecture, memory systems, accelerators, and CIM. Figure 4 shows the domain mix of the final challenge suite.

For each challenge we configure the verifier and its simulator ahead of time so that they can run the design the agent delivers, and we prepare a baseline solution to compare against. We use the baseline-normalization rule and hard-failure handling defined in Section 3.4: values above $1.0\times$ beat the challenge baseline, and invalid or constraint-violating designs are reported as hard failures alongside the baseline-normalized performance aggregates.

We also evaluate how the agent works, using the trajectory rubric defined in Table 6. We record the agent’s visible rationale when available, its actions, and its interactions with the environment, including the version history of its file edits and the detailed output of every simulation run. The trajectory analysis checks process properties aligned with the capability map: task compliance, simulator/tool use, workload-grounded design, performance judgment, constraint awareness, and integrity or originality.

The agent runner is intentionally simple: a lightly modified Mini-SWE framework manages the workspace and tool loop, while our connector tools expose the simulator-facing actions. GPT-5.5 uses the codex CLI backend; the other agent configurations use the same Mini-SWE-style framework. We evaluate four agent configurations

Table 10: L3 exposes the largest agent separation. Values report baseline-normalized performance; 1.0× matches the challenge baseline and larger is better. Geomean/median use valid verifier submissions, while win rate counts all 20 challenges. Arrows mark the better direction; *geo.*, *med.*, and *win* abbreviate geomean, median, and win rate.

Agent		L1			L2			L3			Cross-setting comparison	
Model	Framework	geo.↑	med.↑	win↑	geo.↑	med.↑	win↑	geo.↑	med.↑	win↑	L2>L1↑	L3> {L1, L2} ↑
GPT-5.5	Codex	1.74	1.30	85%	1.88	1.30	95%	1.21	1.15	65%	65%	10%
Gemini 3.5 Flash	MiniSWE	1.68	1.33	85%	1.90	1.63	85%	0.61	0.90	25%	61%	6%
Gemma 4 31B	MiniSWE	1.75	1.32	85%	1.32	1.30	60%	0.72	0.99	35%	47%	6%
Gemini 3.1 Flash-Lite	MiniSWE	1.00	1.02	60%	0.73	0.92	35%	0.45	0.81	30%	12%	21%

that span a wide range of capability and cost: Gemma 4 31B, GPT-5.5, Gemini 3.5 Flash, and Gemini 3.1 Flash-Lite. Gemma 4 31B and GPT-5.5 expose visible rationale traces in our runs. The two Gemini configurations do not expose such traces in this setup; for Gemini 3.5 Flash, our inference setup did not provide access to a rationale-mode API. This distinction matters when interpreting trajectory-based labels and failure modes. We therefore compare complete agent configurations rather than isolated base models. Unless stated otherwise, results in this section use one run per agent, challenge, and evaluation setting.

5.2 Cross-Setting Results

Cross-setting performance overview. This experiment asks how agent performance changes across the three evaluation settings. We run the same 20 challenges for each agent under L1, L2, and L3, using the same verifier baselines and baseline-normalized performance. Table 10 reports the suite-level performance and cross-setting comparisons, and Figure 5 visualizes the same trends.

The table shows two patterns. First, most agents perform well in L1, and all reach roughly baseline or better when the benchmark provides the most help and allows repeated verifier-simulator feedback (geomean 1.00 to 1.75, win rate 60 to 85%). Second, performance drops sharply in L3, where the agent must submit after its own analysis and performance prediction. GPT-5.5 holds up best, with a geomean of 1.74, 1.88, and 1.21 from L1 to L3 and a win rate of 85, 95, and 65%. The other three slide toward or below baseline: Gemini 3.5 Flash falls from 1.68 to 0.61 (after peaking at 1.90 at L2), Gemma 4 31B falls from 1.75 to 0.72 (win rate 85 to 35%), and Gemini 3.1 Flash-Lite falls from 1.00 to 0.45 (60 to 30%).

The central message is that removing simulator feedback separates optimizers from architecture judges. Agents look strong when the full harness supplies repeated simulator feedback, but they separate sharply when they lose simulator access and must rely on static workload evidence, constraints, and their own performance prediction. This suggests an interesting pattern: current LLM agents often behave more like simulator-dependent operators than architecture designers with reliable internal models of performance. This is the core ArchEval signal: many agents can produce runnable architecture artifacts, but without feedback they often cannot judge whether those artifacts are better. The L3 prediction analysis in Section 5.5 gives the detailed evidence.

5.3 L1: Full-Harness Optimization

L1 optimization with the full harness. This experiment isolates what agents can do when the benchmark supplies the full harness and repeated simulator feedback. For each L1 run, we replay verifier-simulator feedback events in order and keep the best valid relative performance seen so far. Figure 6 shows whether repeated simulator feedback actually improves the design, and Table 11 gives a concrete trajectory where GPT-5.5 + Codex uses feedback to improve an 8-NPU AllReduce design. Most of the gain comes in the first several feedback rounds, and the agents that start near the baseline gain the most.

Agents can optimize architecture performance when the full harness turns architecture design into a focused hypothesis-testing loop. Table 11 illustrates that, inside a provided feedback loop, the agent can interpret a bad measurement and choose a qualitatively different next candidate. Compared with a manual sweep, the agent keeps the loop active without requiring a human to inspect every run. The harness also isolates skills that current agents are weak at: building the simulator, defining the metric, and predicting performance before feedback.

The challenge is to optimize an 8-NPU AllReduce on a ring network. The run first tries halving-doubling because it has fewer logical steps, but the simulator shows that the design is much slower on the physical ring. The written notes explain the problem as long-distance traffic; the later candidates return to direct AllReduce and tune chunking. The best measured design reduces total cycles from 15,775 to 10,188, a 35.4% reduction.

5.4 L2: Simulator-Code Container

L2 asks a narrower question than the performance table: when the full design harness is removed but the simulator-code container remains available, how does the agent use the simulator? This is different from L1 design-space exploration, where the benchmark already provides the submission path and measured feedback loop. It is also different from L3 performance prediction, where the agent must submit without simulator feedback. L2 isolates a practical architecture skill: turning simulator access into useful local experiments. L2 is not simply harder than L1: it removes the prepared verifier loop but exposes simulator internals. For weaker agents this creates friction, while for stronger agents it can unlock better local search.

L2 simulator-use behavior. This analysis asks what agents actually do with simulator access before their first submission. We inspect the L2 MiniSWE trajectories for file reading, design editing,

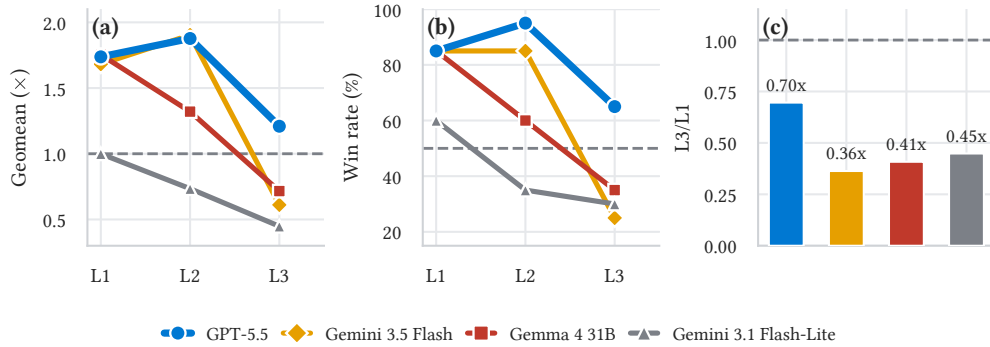


Figure 5: Performance drops most sharply when agents lose simulator feedback. (a) geomean baseline-normalized performance, (b) win rate, and (c) L3/L1 retained geomean.

Table 11: GPT-5.5 + Codex conducts effective design-space exploration for an 8-NPU collective-communication design task. The task is to reduce AllReduce cycles on a ring network; the run uses simulator feedback to reject a lower-step algorithm and then tune direct chunking. The feedback comes from verifier submissions; the reflection is from agent-written notes and the visible run log.

Order	Design	Cycles↓	Agent Reflection	Next design
1	Direct AllReduce baseline.	15,775 cycles.	Need compare algorithm families rather than blindly tune fields.	Try halving-doubling with local-bandwidth-aware scheduling.
2	Halving-doubling with zero endpoint delay.	Design failed: simulator assertion.	Zero endpoint delay likely creates same-time events in this backend.	Keep halving-doubling but use a minimal positive endpoint delay.
3	Halving-doubling with 1 ns endpoint delay.	64,772 cycles.	Fewer logical steps lose because long-distance traffic dominates on the ring.	Return to direct; keep endpoint delay at 1 ns.
4	Direct with 1 ns endpoint delay.	15,748 cycles.	Direct is stable, but endpoint tuning gives only a small gain.	Add 4 payload splits with 2 active chunks.
5	Direct with 4 splits and 2 active chunks.	12,376 cycles.	Chunking is the large remaining win; test more overlap before budget ends.	Increase to 8 splits and 4 active chunks.
6	Direct with 8 splits and 4 active chunks.	10,188 cycles.	Aggressive chunking is best among tested candidates.	Use this as the final design.

build attempts, and simulator runs; Table 12 reports the share of the 20 L2 challenge runs where each action appears. This is a process analysis, not an outcome table: it asks whether the agent exercised the simulator path exposed by L2. The GPT-5.5 Codex runs are excluded from this table because their local shell and file actions are not recorded in the same MiniSWE trace stream; they remain in the outcome tables.

Most agents treat simulator code as static reference material; Codex can turn simulator access into local experiments. The MiniSWE agents usually inspect the simulator workspace and often edit the design, so their failures are not simply blank submissions or unchanged starter code. The gap appears at the simulator step. Gemini 3.5 Flash runs the simulator before its first submission in 60% of L2 runs, but Gemma 4 31B does so in only 30% and Gemini 3.1 Flash-Lite in 15%. This suggests that many L2 submissions are still closer to edit-and-submit behavior than to a local architecture design loop that tests a hypothesis, reads the simulator output, and revises the design.

Codex provides the contrast. In successful L2 runs, it does not only inspect the simulator code; it builds small experiments around the simulator and uses the measurements to change the design. In a CIM accelerator design challenge using MNSIM (MNSIM-CIM),

Table 12: L2 tool-use analysis for MiniSWE agents. Columns report the share of 20 L2 challenge runs in which the trajectory shows the action before the first submission. Build means an explicit local build command; some simulators do not require one. GPT-5.5 + Codex is excluded because its local shell and file actions are not recorded in the same MiniSWE trace stream.

Model	Files↑	Edits↑	Build↑	Sim↑
Gemini 3.5 Flash	100%	95%	30%	60%
Gemini 3.1 Flash-Lite	100%	75%	20%	15%
Gemma 4 31B	100%	95%	15%	30%

the L1 run used the verifier-submission loop and ended at parity with the baseline: larger crossbars improved raw EDP but failed the LeNet [26] area gate, so the best legal result remained 1.00× baseline. In L2, Codex wrote a local sweep.py, generated many temporary simulator configs, and used those runs to find a legal configuration with a 4-bit ADC choice, higher converter parallelism, and a different PE shape; the final relative performance rose to 18.75×. In a systolic-array mapping challenge using SCALE-Sim, Codex’s L1 heuristic chose 4-core output-stationary execution and

Table 13: Agents are weak at both submitted-design self-evaluation and agent-written performance prediction. Self-evaluation columns summarize whether the agent predicts the performance of its submitted design, how accurate that prediction is, and whether it is overconfident. Agent-written model columns evaluate the executable performance models that agents write to compare saved candidate designs; the final column reports the share of challenges with an executable, design-sensitive model, at least three verifier-measured candidates, and $\tau \geq 0.6$.

Model	Agent Framework	Submitted-design self-evaluation				Agent-written performance model			
		Gives self-eval $\uparrow$	Error $\downarrow$	Range hit $\uparrow$	Overestimate ratio $\downarrow$	Executable model $\uparrow$	Within-session agreement $\uparrow$	Cross-session agreement $\uparrow$	Modeling pass rate $\uparrow$
GPT-5.5	Codex	20/20	93%	15%	65%	18/20	+0.20	+0.32	15%
Gemini 3.5 Flash	MiniSWE	9/20	81%	22%	100%	3/20	n/a	-0.67	0%
Gemini 3.1 Flash-Lite	MiniSWE	17/20	96%	0%	82%	2/20	n/a	-0.05	0%
Gemma 4 31B	MiniSWE	16/20	99%	12%	100%	13/20	+0.13	+0.18	5%

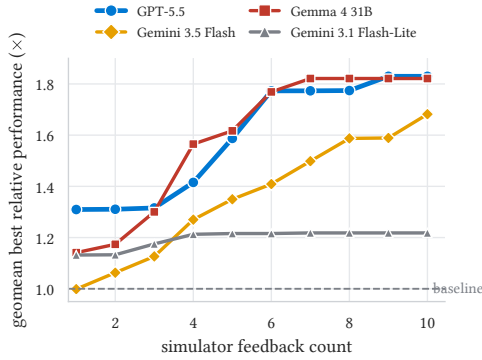


Figure 6: L1 agents improve with repeated verifier-simulator feedback. For each agent–challenge run we replay verifier-simulator feedback events and track the best valid relative performance so far. Each line is the geometric mean over a fixed cohort of runs whose first simulator result is already valid (14 to 20 of 20 per agent), so the best-so-far average rises monotonically with the simulator-feedback count.

lost to the baseline (0.80 $\times$). In L2, simulator probes showed that 4-core policies were slower than expected and that 1-core input-stationary execution gave the best measured result (1.58 $\times$). These cases explain why L2 can beat L1 for a strong agent: code access lets the agent build its own experiment harness instead of only consuming the benchmark’s verifier submissions.

5.5 L3: Agent-Only Prediction Before Feedback

L3 performance prediction. This experiment asks whether agents can evaluate their own design before the verifier simulator provides feedback. In each L3 challenge, the agent must build an agent-written performance model, use it to compare candidate designs, and submit a final self-evaluation. Table 13 summarizes both submitted-design self-evaluation and agent-written model agreement with the verifier simulator; Table 14 shows concrete cases where the agent predicts a win but the verifier measures a loss. We separate two questions: whether the agent can predict its submitted design, and whether its saved performance model ranks alternatives correctly.

For the submitted design, we compare the agent’s own predicted metric and uncertainty range against the verifier simulator. This submitted-design self-evaluation is the agent’s own claim before verifier feedback; it may come from its surrogate model, a hand calculation, or a qualitative estimate, and we do not infer the source. For the saved candidate designs, we test whether the agent-written performance model places designs in the same order as the verifier simulator does.

The agreement columns use Kendall’s τ , a rank-correlation coefficient: +1 means the agent’s model orders candidate designs exactly like the verifier simulator, 0 means no useful ordering, and -1 means the order is reversed. Within-session agreement is computed inside one challenge, between the agent’s surrogate predictions and verifier-simulator measurements for that same agent’s saved candidate designs, for challenges where at least three candidates can be measured. Cross-session agreement asks a coarser question: across challenges, do the agent’s predicted improvements put easy and hard tasks in the same order as the verifier simulator? The performance-modeling pass rate in Table 13 is not another correlation. A challenge passes only if the agent saves enough candidate designs, writes an executable and design-sensitive performance model, obtains at least three verifier measurements, and the model ranking reaches $\tau \geq 0.6$.

Even frontier agents cannot model architecture performance reliably. Submitted-design self-evaluations are inaccurate across all agents: the median relative error is 81–99%, and only 0–22% of predictions contain the verifier result in the agent’s own uncertainty range. Agent-written performance models do not solve the problem. GPT-5.5 + Codex produces an executable performance model on 18 of 20 challenges, and MiniSWE + Gemma 4 31B on 13, but their median within-session agreement with the verifier simulator is only +0.20 and +0.13. Folding model delivery and ranking quality together, the performance-modeling pass rate remains low: 15% for GPT-5.5 + Codex, 5% for MiniSWE + Gemma 4 31B, and 0% for either MiniSWE + Gemini agent. The failure is not merely artifact generation. GPT-5.5 produces executable performance models on 18 of 20 challenges, yet those models weakly agree with the verifier; the missing capability is calibrated architecture judgment.

The errors are not symmetric: agents usually overestimate their own designs. Among L3 runs with a computable submitted-design self-evaluation, agents overestimate the submitted design in 65%

Table 14: LLM agents often overestimate runnable designs. Each example runs successfully and satisfies the hard constraints, but the agent predicts a win and the verifier measures a loss.

Agent		Verifier simulator	Agent prediction		Verifier simulator	
Model	Framework		Design	Baseline	Design	Baseline
Gemini 3.1 Flash-Lite	MiniSWE	ASTRA-sim (cycles ↓)	25.8k cycles	26.0k cycles	218.5k cycles	5.24k cycles
Gemini 3.1 Flash-Lite	MiniSWE	Timeloop (EDP ↓)	1.0×10^6 EDP	2.0×10^6 EDP	2.30×10^9 EDP	3.37×10^8 EDP
GPT-5.5	Codex	DRAMSys (GB/s ↑)	41.8 GB/s	38.6 GB/s	62.9 GB/s	72.5 GB/s

of GPT-5.5 runs, 82% of Gemini 3.1 Flash-Lite runs, and 100% of Gemini 3.5 Flash and Gemma 4 31B runs. The final artifacts show the same pattern. Across the 80 L3 agent-challenge runs, 56 final artifacts run successfully in the verifier simulator and satisfy the hard constraints, but 31 of those 56 runnable designs are below baseline, and 19 are cases where the agent predicted an improvement but the verifier simulator measured a loss.

Table 14 gives concrete examples of agent prediction versus verifier-simulator ground truth. In each row, the agent predicts that its design beats the baseline, but the verifier simulator measures the opposite. These comparisons show where the agent’s performance model misses an architecture cost: communication distance, layer-level mapping, or memory-controller behavior. The per-stage funnel behind Table 13 is in Table 24, and the per-challenge Kendall- τ values are in Table 26.

The examples fail for different architecture reasons. In ASTRA-sim AllToAll, the surrogate treats the ring all-to-all choice as nearly neutral, while the verifier exposes severe communication cost. In Timeloop, the agent-written performance model misses layer-level mapping cost by orders of magnitude. In DRAMSys, the queueing model predicts a bandwidth gain, but the verifier simulator measures a slower memory-controller configuration.

Taken together, the three setting-level experiments show where support matters: L1 demonstrates assisted optimization, L2 tests whether source access becomes a working experimental loop, and L3 exposes the gap between runnable design generation and calibrated performance judgment. Section 6 turns these setting-level results into cross-cutting capability findings.

6 Capability Findings: Can LLM Agents Act as Architects?

The previous section organizes the empirical results by evaluation setting. This section cuts across the same runs to ask which architecture capabilities current LLM agents actually demonstrate. We keep these findings separate from the setting-level experiments because they are not new protocols; they are diagnoses drawn from trajectories, submitted artifacts, and verifier outcomes.

6.1 Can LLM Agents Use Workload Analysis to Guide Design?

Workload analysis is the architecture step between reading a task and choosing a design. The agent must identify what property of the workload matters, such as branch entropy, cache reuse, collective

Table 15: Many workload analyses are written but not grounded or used. For the 17 L3 challenges where we ask agents to produce a workload-analysis document, *Produces* is the share that delivered one. *Grounded* means the analysis reports characteristics measured from the actual workload rather than generic intuition; *Guides design* means the submitted design implements the analysis hypothesis. Grounded and Guides design are labeled by Gemma 4 31B.

Model	Produces↑	Grounded↑	Guides design ↑
GPT-5.5	100%	82%	94%
Gemma 4 31B	88%	53%	53%
Gemini 3.5 Flash	41%	29%	35%
Gemini 3.1 Flash-Lite	94%	41%	53%

communication pattern, or layer-level data reuse, and then choose a design that responds to that property. We evaluate this skill at L3, where the agent cannot rely on iterative simulator feedback and must use its own analysis to decide what to submit.

Workload-analysis design use. This analysis asks whether a written workload analysis changes the submitted architecture design. Table 15 checks the L3 trajectories: whether the agent produced the requested analysis, whether that analysis is grounded in workload-specific evidence, and whether the submitted design follows the analysis. The last two labels are produced by Gemma 4 31B. Table 16 removes the judge from the loop with a controlled ASTRA-sim comparison, where five tasks differ only in the collective workload and the submitted design specification reveals which collective algorithm the agent chose.

Finding 1: Many agents produce workload analyses that are not grounded in the workload or not reflected in the design.

GPT-5.5 is the exception: it produces the analysis on every relevant task, grounds 82% of them in workload-specific evidence, and its design follows the analysis 94% of the time. The other agents often produce generic discussion that does not change the architecture choice. Gemini 3.1 Flash-Lite writes an analysis in 94% of cases, but only 41% are grounded, and in the ASTRA-sim collective challenges it still submits ring for every workload. Gemini 3.5 Flash fails earlier, producing an analysis on only 41% of the relevant tasks. The important signal is whether the workload evidence changes the submitted design.

Table 16: Does the design follow the workload? A natural experiment on the ASTRA-sim collective challenges: five versions of the same task that differ only in the collective-communication workload (L3). Each entry is the collective algorithm the agent encoded in the submitted design. GPT-5.5 picks a workload-appropriate algorithm for each collective; the other three submit `ring` for every workload, so their design never changes with the workload.

Collective workload	GPT-5.5	Gemma 4 31B	Gemini 3.5 Flash	Gemini 3.1 Flash-Lite
All-gather	halving-doubling	ring	ring	ring
All-to-all	direct	ring	ring	ring
All-to-all, 16 nodes	direct	ring	ring	ring
Reduce-scatter	ring	ring	ring	ring
All-reduce	direct	ring	ring	ring

6.2 Can LLM Agents Judge Performance Before Feedback?

The L3 experiment in Section 5.5 isolates the central architecture judgment problem: the agent must decide which feasible design is worth submitting before the verifier simulator gives feedback. Table 13 shows that current agents are weak at this skill even when they deliver the requested artifacts.

Finding 2: Even frontier agents cannot model architecture performance reliably.

Submitted-design self-evaluations are inaccurate across all agents: the median relative error is 81–99%, and only 0–22% of predictions contain the verifier result in the agent’s own uncertainty range. Agent-written performance models do not solve the problem. GPT-5.5 + Codex produces an executable performance model on 18 of 20 challenges, and MiniSWE + Gemma 4 31B on 13, but their median within-session agreement with the verifier simulator is only +0.20 and +0.13. Folding model delivery and ranking quality together, the performance-modeling pass rate remains low: 15% for GPT-5.5 + Codex, 5% for MiniSWE + Gemma 4 31B, and 0% for either MiniSWE + Gemini agent.

Finding 3: Agents usually overestimate their own designs.

The main failure mode is not a blank submission or an inability to write code. It is overconfident architecture judgment. Across the 80 L3 agent–challenge runs, 56 final artifacts run successfully and satisfy the hard constraints, but 31 of those 56 runnable designs are below baseline. Nineteen are cases where the agent predicted an improvement and the verifier simulator measured a loss. This is the gap that separates a useful optimization assistant from an autonomous architect: the agent can often produce a feasible mechanism, but it cannot reliably tell whether that mechanism is better before feedback.

6.3 Can LLM Agents Follow Resource Constraints While Making Design Trade-offs?

Architecture design is not just maximizing one metric. A useful design must stay inside resource limits such as storage budgets,

Table 17: MiniSWE + Gemini 3.1 Flash-Lite satisfies a strict 256 B SRAM metadata budget. The example is an L3 LLC replacement-policy task where the agent must reason about physical storage, not C++ heap state.

Task	Design an LLC replacement policy for a 2048-set, 16-way cache under a total 256 B metadata budget.
Agent note	“Given the extremely tight budget of 256 bytes for 2048 sets, we must use a very compact representation.”
Submitted calculation	“2048 sets * 1 bit = 2048 bits = 256 bytes. So I can have 1 bit per set.” The design then declares <code>std::array<uint8_t, 256> state</code> .
Verifier result	The storage checker reports 2048 bits = 256 B, no dynamic-allocation warnings, and PASS. The measured IPC is 0.354 versus the 0.3747 LRU baseline.

Table 18: GPT-5.5 most consistently returns the requested artifacts. Task compliance means producing the required deliverables in the requested format, independent of whether the design is high-performing.

Agent		L1↑	L2↑	L3↑
Model	Framework			
GPT-5.5	Codex	100	100	94
Gemma 4 31B	MiniSWE	62	70	80
Gemini 3.5 Flash	MiniSWE	25	40	38
Gemini 3.1 Flash-Lite	MiniSWE	85	90	83

Table 19: Constraint following (%). Share of submissions that respect the challenges hard limits (storage budget, area, accuracy, topology, format), by evaluation setting.

Agent		L1↑	L2↑	L3↑
Model	Framework			
GPT-5.5	Codex	75	80	90
Gemma 4 31B	MiniSWE	70	72	63
Gemini 3.5 Flash	MiniSWE	90	83	85
Gemini 3.1 Flash-Lite	MiniSWE	50	59	47

area caps, topology constraints, accuracy floors, and format requirements. We therefore analyze whether each final artifact respects the hard constraints that make the design trade-off meaningful.

Constraint following. This analysis separates feasibility from performance. We inspect whether final artifacts satisfy hard resource and format constraints, then use Table 17 as a concrete example of a strict 256 B SRAM metadata budget. The process tables report task compliance and hard constraint following across settings.

Finding 4: Precise, program-checkable constraints help agents produce feasible designs, but feasibility does not imply good design judgment.

Constraint following is a strength of current frontier agents when the constraint is stated in a precise, program-checkable form. In the ChampSim cache-replacement task, the resource limit is deliberately hardware-like: the whole replacement policy may use

Table 20: LLM agents are very weak in design novelty. On the six code-authoring challenges, a submission counts as novel only if it goes beyond recombining ≤ 3 known policies (LRU, TAGE, stride, tournament, ...) with parameter tuning. Entries show novel / total; only GPT-5.5 + Codex produces one novel design.

Agent		L1↑	L2↑	L3↑	Total↑
Model	Framework				
GPT-5.5	Codex	0/6	0/6	1/6	1/18
Gemma 4 31B	MiniSWE	0/6	0/6	0/6	0/18
Gemini 3.5 Flash	MiniSWE	0/6	0/6	0/6	0/18
Gemini 3.1 Flash-Lite	MiniSWE	0/6	0/6	0/6	0/18

only 256 B of SRAM metadata. This is a total budget for the candidate object, not a per-set allowance. Because the LLC has 2048 sets and 16 ways, the budget leaves only 2048 bits across 32,768 cache lines. A normal C++ implementation could hide extra state in heap allocation or large STL containers, but that would not correspond to a physical SRAM budget. The verifier therefore runs a static storage checker that rejects hidden dynamic state and counts the hardware bits represented by the submitted fields.

The cache-replacement example in Table 17 shows the pattern directly. Even a smaller agent can parse and satisfy a hardware-style storage constraint when the verifier makes the rule precise. The submitted design still underperforms LRU, which is the point: current agents can often satisfy explicit resource constraints, but they still struggle to choose the best feasible design. The task-compliance analysis is mainly a sanity check: modern code agents are often able to produce the requested files and formats (Table 18), especially for the stronger model. The more architecture-specific question is whether those files encode a valid design under the hard resource rules. In the process analysis, GPT-5.5 respects the hard constraints in most submissions, including 90% of L3 submissions (Table 19).

This example highlights a useful kind of agent capability: exact bookkeeping over many small hardware constraints. Human designers routinely make these trade-offs, but checking every hidden byte of state is tedious and error-prone; an agent can apply the rule mechanically once the benchmark exposes it in a precise form.

6.4 Can LLM Agents Propose Novel Designs?

Design novelty. This analysis asks whether agent improvements come from new architecture mechanisms or from recombining familiar human designs. We label the delivered designs and trajectories by whether they go beyond known policies and light parameter tuning; Table 20 summarizes the result, while the case study below examines the only design that clears the novelty bar.

Finding 5: In audited code-authoring challenges, agents show little evidence of genuine mechanism discovery.

Across the analyzed trajectories, most improvements come from composing existing human design patterns and lightly tuning their parameters, rather than proposing new mechanisms. This is not always a failure, since good architecture often reuses known ideas in the right place, but it means current agents are still much stronger at recombination than at original architecture design. This finding also

Table 21: Gemini 3.5 Flash submits the starter as the final design on two L3 source-code tasks. We compare the final submitted source files against the provided starter files byte-for-byte.

L3 source-code task	Starter copied?
Branch predictor	Yes
BTB	No
Cache replacement	No
Branch predictor + BTB	No
Repl. + prefetch	Yes
L1D prefetcher	No

explains why the L3 setting matters: without repeated simulator feedback, agents must choose which design ideas are worth trying, and current agents mostly draw from familiar human-written templates instead of expanding the design space.

Case study: the lone novel policy, and why novelty alone is not enough. The single design our judge flags as novel (Table 20: GPT-5.5 on cache replacement at L3) abandons recency entirely. In place of an LRU stack, its `find_victim` hashes a combination of the resident line address, the incoming line address, the requesting instruction pointer, and the set index through a `splmix64`-style integer mixer, and evicts the way with the largest resulting priority—a deterministic, pseudo-random choice keyed on access *context* rather than on recency. It layers two further hand-rolled heuristics on top: a same-region bias that adds a large constant to the eviction priority when a resident line shares the incoming line’s address region (preferring to evict spatial neighbors), and a one-bit-per-set “hot set” tracker that, for sets it marks hot, restricts eviction to the upper half of the ways so the lower half is protected. None of these mechanisms corresponds to a textbook replacement policy (LRU, RRIP, LFU, ...), which is why it is the only submission in the suite that clears the novelty bar.

The design does not pay off: it reaches only $0.91\times$ the LRU baseline—it *loses*. Pseudo-random victim selection discards precisely the recency signal that LRU exploits, and the region and hot-set heuristics do not recover it. This sharpens the performance-prediction diagnostic: across the 72 code-authoring submissions an agent proposed a genuinely new mechanism exactly once, and that once it was a net regression. Current agents are therefore not merely biased toward recombining known policies; on the one occasion an agent stepped outside the known set, the result was worse than reusing a standard policy well. This case illustrates that novelty alone is insufficient without reliable performance judgment. *Single-seed (s1); novelty labeled by Gemma 4 31B.*

6.5 Shortcut and Interface Failure Modes

Shortcut behavior at the interface. Novelty asks whether the design expands the mechanism space; integrity asks whether the submitted artifact actually represents the claimed mechanism. This diagnostic asks whether an agent can obtain a superficially valid submission without doing the intended architecture work. We distinguish these interface shortcuts from direct hidden-answer leaks, which the platform blocks by design. Table 21 reports one measurable pattern: final submissions that are byte-identical to the provided starter.

Finding 6: Trajectory logs expose shortcut behavior that final performance alone would hide.

We use shortcut behavior to mean behavior that makes a submission look successful through an unintended interface path rather than by solving the architecture design task. Direct shortcuts, such as reading a hidden reference or leaking verifier state, are blocked by the platform. In the final runs, we still observe two weaker but important forms of shortcut behavior.

The first form is starter-as-final behavior. In these runs, the agent submits the starter design provided in the task environment as the final answer. For Gemini 3.5 Flash, two of the six L3 source-code tasks are byte-identical to the starter (Table 21).

The second form is a mismatch between the claimed mechanism and the submitted implementation. In one Gemma 4 31B branch-predictor run, the design document states that the chosen design is a Gshare predictor with one global-history table. The submitted code instead implements a tournament predictor: a bimodal table, a Gshare table, and a meta-predictor that chooses between them. This is not a hidden-reference leak, but it is still a benchmark risk: the artifact can improve the metric while the written design argument no longer describes the mechanism that was actually evaluated. The remaining process tables summarize trajectory-rubric dimensions used across the analyses above.

Taken together, these diagnostics show why a single performance number is insufficient. L1 demonstrates that agents can make use of the full harness. L2 shows that simulator access is useful only when the agent can turn source into experiments. L3 shows that runnable artifacts and explicit constraint satisfaction do not imply reliable architecture judgment. The remaining gap is not artifact generation alone, but grounded performance prediction and choosing the right feasible mechanism before simulator feedback.

7 Related Work

Section 2 frames AI architecture work as prediction, optimization, and generation under changing amounts of support. Related work covers important parts of this space, but usually fixes one axis: software-agent benchmarks provide executable tasks outside architecture, DSE systems evaluate optimization inside a prepared loop, and hardware-generation benchmarks focus on RTL artifacts. We organize the comparison around five requirements for an architecture-agent benchmark: architecture-specific design artifacts, simulator-backed execution, multi-domain coverage, controlled variation in experimental support, and trajectory-level diagnostics. Table 5 summarizes how ArchEval combines these requirements.

7.1 Agentic Execution-Grounded Benchmarks

Following the reasoning-and-acting paradigm, LLM agents combine a language model with a harness that gives the model tools, observations, and a multi-step workflow [51]. Several recent benchmarks evaluate this agentic setting with real execution. SWE-bench asks an agent to resolve a real GitHub issue and checks the fix against the repository’s tests [18]. MAgentBench evaluates agents on machine-learning experimentation workflows [17]. Terminal-bench evaluates agents on hard end-to-end tasks in a command-line

environment [31]. ProgramBench asks an agent to rebuild a program from its executable and documentation, then evaluates the result with behavioral tests [50]. MAgentBench is especially close in spirit because it evaluates iterative experimentation, but the workflow is machine-learning experimentation rather than architecture simulation, and it does not vary how much of the design/evaluation loop is supplied.

These benchmarks establish the value of evaluating agents through actions, tools, trajectories, and executable outcomes. Their artifacts, however, are software patches, ML experiments, terminal workflows, or programs judged by behavioral tests. Performance-code benchmarks such as KernelBench and CUDA-agent-style systems similarly evaluate generated artifacts by execution and speed [7, 33]. They are close in spirit to ArchEval’s executable scoring, but their artifacts are software kernels rather than architecture mechanisms, mappings, or simulator-level design choices evaluated under architecture constraints.

7.2 Architecture Knowledge and Design-Space Exploration

Other work is specific to computer architecture, but evaluates a narrower capability. QuArch tests architecture knowledge through question answering, covering processor design, memory systems, and interconnects [37]. It is architecture-specific, but it does not ask an agent to produce a design and have that design measured by a simulator.

Classical and learning-based DSE frameworks make architecture optimization programmable: they expose simulators, design spaces, and objective functions so search algorithms can improve a metric [22, 24, 52]. These frameworks are essential infrastructure for architecture optimization, but they assume that the design variables, legal choices, simulator path, and feedback loop are already defined. Recent LLM-for-architecture systems build on this setting by placing a language model inside the optimization loop [4, 12, 45, 47]. These systems primarily evaluate the model inside a simulator-driven search loop: given a simulator, a design interface, and a largely predefined search space, the agent searches for a better configuration or design candidate. ArchEval treats this as an L1-like capability, then asks what happens when the prepared loop is removed or when simulator feedback is unavailable.

This distinction is methodological as well as domain-specific. Most prior agent and DSE evaluations fix the amount of experimental support. ArchEval varies that support while holding the task and verifier fixed, separating feedback-loop optimization from simulator-tool use and pre-feedback design judgment.

7.3 Hardware Generation and EDA Agents

A related hardware-design line evaluates LLMs on HDL and EDA tasks. VerilogEval tests Verilog generation against functional testbenches [27]; AutoChip uses compiler and simulation feedback to improve HDL generation [43]; and Chip-Chat explores conversational hardware design through an interactive HDL tapeout case study [5]. These works are closer to hardware than software-agent benchmarks, and they show that execution feedback is useful for hardware artifacts. In these settings, the central question is whether

the generated hardware description satisfies functional tests, synthesis or tool constraints, or an HDL design workflow. In ArchEval, the central question is whether an architecture mechanism, mapping, or configuration improves workload-level performance under a simulator-backed baseline and hard resource constraints.

7.4 Summary: Benchmark Gap

Prior work therefore evaluates agents without architecture, architecture knowledge without executable design, architecture optimization under fixed support, hardware generation at the RTL or tool level, or performance prediction as a standalone model. ArchEval targets the benchmark gap between these lines. It combines architecture-specific artifacts, simulator-backed verification, multi-domain coverage, controlled L1/L2/L3 support levels, and trajectory diagnostics. It asks whether an agent finds a high-performing design and which parts of the architecture loop produced that point: workload interpretation, simulator-tool use, feedback-driven optimization, pre-feedback performance judgment, constraint handling, and artifact integrity.

8 Discussion

The results above suggest a practical but incomplete role for current architecture agents. They are useful when humans provide the design interface, simulator, metric, and feedback loop, but they are not yet reliable substitutes for early-stage architecture judgment. This section translates the ArchEval diagnostics into usage guidelines, open research questions, and limitations of the current benchmark.

8.1 Best Practices for Architecture Agents

The lesson is to use agents as optimization assistants inside accountable workflows, not as unsupervised architecture designers.

Use agents inside a prepared verifier-simulator loop. The L1 optimization experiment shows the clearest near-term use case: when the architect provides the design interface, metric, verifier simulator, and repeated feedback, agents can act as effective design-space optimizers. This does not mean the agent owns the architecture problem. It means that, after humans frame the design space and measurement loop, the agent can keep proposing variants, reading measurements, and improving the next candidate.

Make workload analysis accountable to the submitted design. The workload-to-design analysis shows that asking for a workload analysis is not enough; most agents can produce generic text that never changes the design. A useful workflow should ask whether the analysis names workload-specific evidence and whether the final architecture choice follows that evidence. In practice, workload analysis should be treated as design evidence, not as a documentation deliverable.

State resource constraints in a program-checkable form. The constraint-following analysis shows that agents can follow tight architecture constraints when the rule is explicit and the verifier checks the physical resource being constrained. This is a practical way to use agents today: give them precise budgets, such as storage, area, topology, or accuracy limits, and use static or simulator-backed checks to make sure the proposed design remains

feasible. Constraint satisfaction should be treated as feasibility evidence, not as proof of good design judgment.

Require a performance model, but verify it against the simulator. Agents should be asked to submit their performance model, predicted metric, and uncertainty estimate along with the design, but those predictions should be treated as testable hypotheses. The L3 diagnostic shows that executable agent-written models often weakly agree with the verifier, and that agents usually overestimate their final designs. A useful workflow should compare the agent’s prediction with simulator results and use the mismatch to refine prompts, constraints, or the model structure itself.

8.2 Open Questions

The open questions fall into two groups: improving the agents themselves, and improving how we measure progress over their trajectories.

Can agents turn simulator source into useful local experiments? The L2 simulator-use analysis shows that simulator access alone is not enough. Many agents read the simulator code as reference material but do not turn it into a local experimental workflow. A central open question is whether agents can reliably discover build entrypoints, identify relevant inputs, construct small probes, distinguish local probes from canonical verifier behavior, parse noisy outputs, debug stale builds or timeouts, and use simulator measurements to revise architecture hypotheses.

Can agents model performance before verifier-simulator feedback? The L3 performance-prediction diagnostic shows the largest gap. Agents often produce executable performance models, but those models weakly agree with the verifier simulator, and agents usually overestimate their final designs. This is the key difference between operating an evaluation loop and doing early architecture design: the agent must decide which design is worth trying before the real simulator gives an answer. The key issue is calibrated uncertainty as well as prediction error: the agent must know when its proxy is too weak to justify a design decision.

How should we measure improvement over an agent’s whole trajectory? This white paper reports final-design baseline-normalized performance and then uses trajectory analysis to explain how the agent got there. A natural next step is to make the trajectory itself a quantitative object. Recent work on trajectory-level evaluation for iterative scientific design argues that endpoint metrics can hide how efficiently an LLM converts feedback into better candidates, and proposes best-so-far area-under-curve metrics and grounding audits as evaluation axes [53]. For ArchEval, L1 and parts of L2 can support best-so-far area-under-curve measurements over simulator attempts and comparisons against classical or domain-specific DSE baselines. L3 has little or no simulator feedback, so its trajectory metric should instead measure whether intermediate predictions, uncertainty estimates, and design rationales are grounded in the final verifier outcome.

Can agents expand the design space instead of recombining known patterns? The novelty analysis suggests that current agents mostly assemble familiar mechanisms and tune parameters. That can be useful, but it leaves open whether LLM agents can propose genuinely new architecture mechanisms that also survive

workload, constraint, and verifier-simulator checks. The goal is useful mechanism discovery, not novelty for its own sake.

Should architecture agents be one model, or a composition of models? Our experiments evaluate complete agent systems, but many failures look decomposable: one module could analyze workloads, another could assemble simulator tools, another could build surrogate models, another could check constraints, and a coordinator could choose mechanisms. This suggests a broader open question for computer architecture design: whether a single frontier model should own the whole loop, or whether a compositional agent made of specialist models can be more reliable. Recent work on compositional generative modeling argues that complex tasks may benefit from combining models rather than relying on one monolith [9]; work on orchestrating small language models raises a related possibility that smaller specialized models could handle parts of the reasoning or verification loop [46]. Multi-agent systems with explicit interactions provide another possible route for composing specialized agents rather than relying on one monolithic architect [38]. For architecture, the hard part is not decomposing the workflow, but recomposing the evidence into one architecture decision.

8.3 Limitations

This study is preliminary. The current results come from a single-seed initial study, and the reported numbers should be read as an early capability map rather than a final leaderboard. Broader model coverage, more samples per agent–challenge setting, and repeated runs under stable decoding settings are needed before drawing fine-grained conclusions about particular agents.

The experiments compare complete agent configurations rather than base models alone. GPT-5.5 uses the Codex backend, while the other agents use a MiniSWE-style harness, so differences reflect both model capability and agent infrastructure.

The benchmark itself also has limits. The simulator set spans several important subfields, but it does not cover all of computer architecture. Baselines vary in source and strength: some come from standard algorithms, others from hand-written reference designs or published systems. The challenges isolate architecture decisions and run within benchmark budgets; they do not cover months-long industrial design campaigns or full tapeout-scale validation. L3 approximates early-stage design by withholding simulator feedback, but it is not the same as a full open-ended architecture research project: the task, deliverable interface, workload evidence, and final verifier are still defined by the benchmark.

In this version, the trajectory rubric uses Gemma 4 31B for semantic questions such as workload grounding, originality, and artifact-rationale consistency, so those judgments should be treated as relative signals that require further calibration. Finally, novelty is difficult to measure. Architecture often advances by recombining known ideas in the right setting, so ArchEval treats originality as process evidence, not as a single outcome number. As tasks become public, future versions should include held-out or newly contributed challenges to reduce contamination and track progress over time.

8.4 Call for Community Contributions

ArchEval is meant to be community infrastructure, and we invite contributions in two forms. The first is new challenges: tasks drawn from papers or practice, paired with credible baselines and runnable verifiers, that widen the suite across more subfields. A useful challenge contribution should include a task card, workload or workload evidence, baseline, hard constraints, canonical verifier, parser, and visibility policy. The second is new agents: systems that can be evaluated under the same evaluation settings and trajectory rubric. A useful agent contribution should include a runnable configuration, tool-interface assumptions, supported L1/L2/L3 settings, and a logging format.

Each contribution turns the data flywheel further. Evaluation produces traces of where agents succeed, where they fail, and how they work through architecture problems. Those traces can guide benchmark design, agent development, and future training data. Community contributions should therefore expand both sides of the benchmark: the architecture tasks that define what must be measured, and the agent systems that test how much of the design loop can be automated.

9 Conclusion

ArchEval makes the question of AI computer architects measurable by turning it into a capability map rather than a single leaderboard result. The benchmark evaluates the same architecture task under L1 full harness, L2 simulator-code container, and L3 agent-only settings, separating assisted optimization from simulator-tool use and from design judgment before simulator feedback. Its multi-simulator platform then makes this protocol executable across CPU cores, memory systems, accelerators, distributed training, and CIM.

The initial results show that current LLM agents have crossed one important threshold: with the full harness, they can improve real architecture designs across diverse simulators. They have not crossed the next thresholds reliably. In the simulator-code container, many agents read the simulator source but struggle to build useful local experiments. In the agent-only setting, they can produce runnable and constraint-satisfying artifacts, but still fail to predict performance reliably or choose the right feasible design. The main lesson is therefore not that today’s agents cannot help architects; it is that their strongest role today is as optimization assistants inside accountable workflows.

The path toward stronger architecture agents runs through the missing capabilities exposed by ArchEval: reliable simulator-tool use, workload-grounded performance prediction, pre-feedback design judgment, and genuine mechanism discovery. By releasing ArchEval as a benchmark and platform, we aim to give the community a shared way to measure those capabilities, diagnose failures, and build the next generation of AI systems that can take on more of the architect’s work.

References

- [1] Yash Akhauri, Bryan Lewandowski, Cheng-Hsi Lin, Adrian N Reyes, Grant C Forbes, Arissa Wongpanich, Bangding Yang, Mohamed S Abdelfattah, Sagi Perel, and Xingyou Song. 2025. Performance prediction for large systems via text-to-text regression. *arXiv preprint arXiv:2506.21718* (2025).
- [2] Christian Bienia, Sanjeev Kumar, Jaswinder Pal Singh, and Kai Li. 2008. The PARSEC benchmark suite: Characterization and architectural implications. In

- Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques*. 72–81.
- [3] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R Hower, Tushar Krishna, Somayeh Sardashti, et al. 2011. The gem5 simulator. *ACM SIGARCH Computer Architecture News* 39, 2 (2011), 1–7.
 - [4] Alexander Blasberg, Vasilis Kypriotis, and Dimitrios Skarlatos. 2026. Agentic Architect: An Agentic AI Framework for Architecture Design Exploration and Optimization. *arXiv preprint arXiv:2604.25083* (2026).
 - [5] Jason Blocklove, Siddharth Garg, Ramesh Karri, and Hammond Pearce. 2023. Chip-Chat: Challenges and Opportunities in Conversational Hardware Design. *arXiv:2305.13243 [cs.AR]* <https://arxiv.org/abs/2305.13243>
 - [6] James Bucek, Klaus-Dieter Lange, and Joakim v. Kistowski. 2018. SPEC CPU2017: Next-generation compute benchmark. In *Companion of the 2018 ACM/SPEC International Conference on Performance Engineering*. 41–42.
 - [7] Weinan Dai, Hanlin Wu, Qiying Yu, Huan ang Gao, Jiahao Li, Chengquan Jiang, Weiqiang Lou, Yufan Song, Hongli Yu, Jiaze Chen, Wei-Ying Ma, Ya-Qin Zhang, Jingjing Liu, Mingxuan Wang, Xin Liu, and Hao Zhou. 2026. CUDA Agent: Large-Scale Agentic RL for High-Performance CUDA Kernel Generation. *arXiv:2602.24286 [cs.LG]* <https://arxiv.org/abs/2602.24286>
 - [8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xi-aohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *arXiv preprint arXiv:2010.11929* (2020).
 - [9] Yilun Du and Leslie Kaelbling. 2024. Compositional generative modeling: A single model is not all you need. *arXiv preprint arXiv:2402.01103* (2024).
 - [10] Michael Ferdman, Almutaz Adileh, Onur Kocberber, Stavros Volos, Mohammad Alisafae, Djordje Jevdjic, Cansu Kaynak, Adrian Daniel Popescu, Anastasia Ailamaki, and Babak Falsafi. 2012. Clearing the clouds: a study of emerging scale-out workloads on modern hardware. *ACM SIGPLAN Notices* 47, 4 (2012), 37–48.
 - [11] Nathan Guber, Gino Chacon, Lei Wang, Paul V Gratz, Daniel A Jimenez, Elvira Teran, Seth Pugsley, and Jinchun Kim. 2022. The championship simulator: Architectural simulation for education and competition. *arXiv preprint arXiv:2210.14324* (2022).
 - [12] Raghav Gupta, Akanksha Jain, Abraham Gonzalez, Alexander Novikov, Po-Sen Huang, Matej Balog, Marvin Eisenberger, Sergey Shirobokov, Ngan Vu, Martin Dixon, et al. 2026. ArchAgent: Agentic AI-driven Computer Architecture Discovery. *arXiv preprint arXiv:2602.22425* (2026).
 - [13] Song Han, Huizi Mao, and William J Dally. 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. *arXiv preprint arXiv:1510.00149* (2015).
 - [14] Milad Hashemi, Kevin Swersky, Jamie Smith, Grant Ayers, Heiner Litz, Jichuan Chang, Christos Kozyrakis, and Parthasarathy Ranganathan. 2018. Learning memory access patterns. In *International Conference on Machine Learning*. PMLR, 1919–1928.
 - [15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 770–778.
 - [16] Charles Hong, Qijing Huang, Grace Dinh, Mahesh Subedar, and Yakun Sophia Shao. 2023. DOSA: Differentiable Model-Based One-Loop Search for DNN Accelerators. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 209–224.
 - [17] Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. 2023. MAgentBench: Evaluating Language Agents on Machine Learning Experimentation. *arXiv preprint arXiv:2310.03302* (2023).
 - [18] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world github issues?. In *International Conference on Learning Representations*, Vol. 2024. 54107–54157.
 - [19] Matthias Jung, Christian Weis, and Norbert Wehn. 2015. DRAMSys: A flexible DRAM subsystem design space exploration framework. *IPSJ Transactions on System and LSI Design Methodology* 8 (2015), 63–74.
 - [20] Yoongu Kim, Ross Daly, Jeremie Kim, Chris Fallin, Ji Hye Lee, Donghyuk Lee, Chris Wilkerson, Konrad Lai, and Onur Mutlu. 2014. Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors. *ACM SIGARCH Computer Architecture News* 42, 3 (2014), 361–372.
 - [21] Yoongu Kim, Weikun Yang, and Onur Mutlu. 2015. Ramulator: A fast and extensible DRAM simulator. *IEEE Computer architecture letters* 15, 1 (2015), 45–49.
 - [22] Srivatsan Krishnan, Natasha Jaques, Shayegan Omidshafiei, Dan Zhang, Izzeddin Gur, Vijay Janapa Reddi, and Aleksandra Faust. 2022. Multi-agent reinforcement learning for microprocessor design space exploration. *arXiv preprint arXiv:2211.16385* (2022).
 - [23] Srivatsan Krishnan, Zishen Wan, Kshitij Bhardwaj, Paul Whatmough, Aleksandra Faust, Sabrina Neuman, Gu-Yeon Wei, David Brooks, and Vijay Janapa Reddi. 2022. Automatic domain-specific soc design for autonomous unmanned aerial vehicles. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 300–317.
 - [24] Srivatsan Krishnan, Amir Yazdanbakhsh, Shvetank Prakash, Jason Jabbour, Ikehukwu Uchendu, Susobhan Ghosh, Behzad Boroujerdi, Daniel Richins, Devashree Tripathy, Aleksandra Faust, et al. 2023. Archgym: An open-source gymnasium for machine learning assisted architecture design. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–16.
 - [25] Alex Krizhevsky, Geoffrey Hinton, et al. 2009. Learning Multiple Layers of Features from Tiny Images. (2009).
 - [26] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-Based Learning Applied to Document Recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324.
 - [27] Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. *arXiv:2309.07544 [cs.PL]* <https://arxiv.org/abs/2309.07544>
 - [28] Xueguang Ma, Qian Liu, Dongfu Jiang, Ge Zhang, Zejun Ma, and Wenhui Chen. 2025. General-Reasoner: Advancing LLM Reasoning Across All Domains. *arXiv:2505.14652 [cs.CL]* <https://arxiv.org/abs/2505.14652>
 - [29] Scott McFarling. 1993. *Combining Branch Predictors*. Technical Report. Technical Report TN-36, Digital Western Research Laboratory.
 - [30] Charith Mendis, Alex Renda, Saman Amarasinghe, and Michael Carbin. 2019. Ithelma: Accurate, portable and fast basic block throughput estimation using deep neural networks. In *International Conference on machine learning*. PMLR, 4505–4515.
 - [31] Mike A Merrill, Alexander G Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E Kelly Buchanan, et al. 2026. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868* (2026).
 - [32] OpenAI. 2026. An OpenAI model has disproved a central conjecture in discrete geometry. <https://openai.com/index/model-disproves-discrete-geometry-conjecture/>. Accessed: 2026-06-16.
 - [33] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. KernelBench: Can LLMs Write Efficient GPU Kernels? *arXiv:2502.10517 [cs.LG]* <https://arxiv.org/abs/2502.10517>
 - [34] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A Ying, Anurag Mukkara, Rangharajan Venkatesan, Bruce Khailany, Stephen W Keckler, and Joel Emer. 2019. Timeloop: A systematic approach to dnn accelerator evaluation. In *2019 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 304–315.
 - [35] Angshuman Parashar, Minsoo Rhu, Anurag Mukkara, Antonio Puglielli, Rangharajan Venkatesan, Bruce Khailany, Joel Emer, Stephen W Keckler, and William J Dally. 2017. SCNN: An accelerator for compressed-sparse convolutional neural networks. *ACM SIGARCH Computer Architecture News* 45, 2 (2017), 27–40.
 - [36] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Gori, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative LLM inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
 - [37] Shvetank Prakash, Andrew Cheng, Arya Tschand, Mark Mazumder, Varun Gohil, Jeffrey Ma, Jason Yik, Zishen Wan, Jessica Quayle, Elisavet Lydia Alvanaki, et al. 2025. Quarch: A benchmark for evaluating llm reasoning in computer architecture. *arXiv preprint arXiv:2510.22087* (2025).
 - [38] Zhenting Qi, Huangyuan Su, Ao Qu, Chenyu Wang, Yu Yao, Han Zheng, Kushal Chattopadhyay, Guowei Xu, Zihan Wang, Weirui Ye, et al. 2026. Economy of Minds: Emerging Multi-Agent Intelligence with Economic Interactions. *arXiv preprint arXiv:2606.02859* (2026).
 - [39] Ritik Raj, Sarbartha Banerjee, Nikhil Chandra, Zishen Wan, Jianming Tong, Ananda Samajdhara, and Tushar Krishna. 2025. SCALE-Sim v3: A modular cycle-accurate systolic accelerator simulator for end-to-end system analysis. In *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 186–200.
 - [40] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, et al. 2020. MLPerf inference benchmark. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 446–459.
 - [41] Hanbo Sun, Chenyu Wang, Zhenhua Zhu, Xuefei Ning, Guohao Dai, Huazhong Yang, and Yu Wang. 2022. Gibbon: Efficient Co-Exploration of NN Model and Processing-in-Memory Architecture. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 867–872.
 - [42] Rajeev Thakur, Rolf Rabenseifner, and William Gropp. 2005. Optimization of Collective Communication Operations in MPICH. *The International Journal of High Performance Computing Applications* 19, 1 (2005), 49–66.
 - [43] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. 2023. AutoChip: Automating HDL Generation Using LLM Feedback. *arXiv:2311.04887 [cs.AR]* <https://arxiv.org/abs/2311.04887>

- [44] Arya Tschand, Chenyu Wang, Zishen Wan, Andrew Cheng, Ioana Cristescu, Kevin He, Howard Huang, Alexander Ingare, Akseli Kangaslahti, Sara Kangaslahti, et al. 2026. GenAI for Systems: Recurring Challenges and Design Principles from Software to Silicon. *arXiv preprint arXiv:2602.15241* (2026).
- [45] Chenyu Wang, Jiahe Caroline Shi, David Kong, Duane S. Boning, Zishen Wan, Yilun Du, and Vijay Janapa Reddi. 2026. AgentDSE: Reasoning-Augmented Architectural Design Space Exploration. *arXiv:2606.21836 [cs.AR]* <https://arxiv.org/abs/2606.21836>
- [46] Chenyu Wang, Zishen Wan, Hao Kang, Emma Chen, Zhiqiang Xie, Tushar Krishna, Vijay Janapa Reddi, and Yilun Du. 2025. Slm-mux: Orchestrating small language models for reasoning. *arXiv preprint arXiv:2510.05077* (2025).
- [47] Hanyu Wang, Xinrui Wu, Zijian Ding, Su Zheng, Chengyue Wang, Neha Prakriya, Tony Nowatzki, Yizhou Sun, and Jason Cong. 2025. LLM-DSE: Searching Accelerator Parameters with LLM Agents. *arXiv preprint arXiv:2505.12188* (2025).
- [48] William Won, Taekyung Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. 2023. Astra-sim2.0: Modeling hierarchical networks and disaggregated systems for large-model training at scale. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 283–294.
- [49] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv:2405.15793 [cs.SE]* <https://arxiv.org/abs/2405.15793>
- [50] John Yang, Kilian Lieret, Jeffrey Ma, Parth Thakkar, Dmitrii Pedchenko, Sten Sootla, Emily McMillin, Pengcheng Yin, Rui Hou, Gabriel Synnaeve, et al. 2026. ProgramBench: Can Language Models Rebuild Programs From Scratch? *arXiv preprint arXiv:2605.03546* (2026).
- [51] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629* (2022).
- [52] Amir Yazdanbakhsh, Christof Angermueller, Berkin Akin, Yanqi Zhou, Albin Jones, Milad Hashemi, Kevin Swersky, Satrajit Chatterjee, Ravi Narayanaswami, and James Laudon. 2021. Apollo: Transferable architecture exploration. *arXiv preprint arXiv:2102.01723* (2021).
- [53] Marilyn Zhang, Tianfeng Chen, Fabián Barzuna, Ankita Rathod, and Mark E. Whiting. 2026. LEAP: Trajectory-Level Evaluation of LLMs in Iterative Scientific Design. *arXiv:2605.15341 [cs.LG]* <https://arxiv.org/abs/2605.15341>
- [54] Zhenhua Zhu, Hanbo Sun, Tongxin Xie, Yu Zhu, Guohao Dai, Lixue Xia, Dimin Niu, Xiaoming Chen, Xiaobo Sharon Hu, Yu Cao, et al. 2023. Mnsim 2.0: A behavior-level modeling tool for processing-in-memory architectures. *IEEE transactions on computer-aided design of integrated circuits and systems* 42, 11 (2023), 4112–4125.

Appendix

A The Challenge Suite

Section A.1 specifies all 20 challenges in ArchEval. Each is posed under all three evaluation settings and measured against its own baseline, where baseline-normalized performance above 1.0× beats the baseline regardless of the metric’s direction. Every challenge includes a task prompt, workload, verifier simulator, deliverable format, hard constraints, and reference baseline design. Most baselines are hand-written reference designs; `cache_replacement` is measured against LRU, and `gibbon_codesign` is measured against the design reported in Gibbon [41].

A.1 Per-Challenge Protocol

Every challenge is posed under all three evaluation settings (L1 full harness / L2 simulator-code container / L3 agent-only) against the same baseline and `evaluate.sh`; the settings differ only in experimental support, source visibility, and verifier-attempt cap. The cap below is not a budget of valid feedback results. In L2 and L3, build failures, validation rejects, and timeouts may be retried up to the cap, but the run stops at the first valid verifier result; the agent does not receive multiple valid verifier measurements for optimization. L1 is the only setting where repeated valid verifier measurements are intended as feedback. The deliverable line below lists the architecture artifact and common design documents; at L3 these are accompanied by agent-written prediction artifacts, such as a surrogate script and/or `prediction.json`, which are evaluated post-session for calibration and agreement with the verifier.

1. Branch Direction Predictor `branch_predictor` (CPU core, *ChampSim*)

Objective: Implement the candidate taken/not-taken predictor class (BTB and pipeline fixed) to minimize average MPKI, following classic branch-prediction design practice [29].

Workload: 3 SPEC CPU2017 traces [6] (`perlbenc`, `gcc`, `xalancbmk`); 3M-instr warmup + 7M-instr measure.

Metric: MPKI ↓ vs stock bimodal. **Constraint:** ≤16 KB SRAM metadata, no dynamic allocation; code ≤1500 lines.

Deliverables: `candidate.{h,cc}`; `workload/principles/evaluation docs`; `pytest surrogate`; `prediction.json`.

Verifier-attempt cap (L1/L2/L3): 10 / 1 / 1.

2. Branch Target Buffer `btb` (CPU core, *ChampSim*)

Objective: Design the candidate BTB minimizing 3-trace average MPKI under a tight metadata budget.

Workload: 3 SPEC CPU2017 traces (`perlbenc`, `gcc`, `xalancbmk`); 3M + 7M instr.

Metric: MPKI ↓ vs stock `basic_btb`. **Constraint:** ≤16 KB metadata; code ≤1500 lines.

Deliverables: `candidate.{h,cc}`; three design docs; `pytest surrogate`; `prediction.json`.

Verifier-attempt cap (L1/L2/L3): 10 / 1 / 1.

3. Cache Replacement Policy `cache_replacement` (CPU core, *ChampSim*)

Objective: Design an LLC replacement policy maximizing cycle-weighted IPC (geomean speedup) over six workloads.

Workload: 6 SPEC CPU2017 traces (`sphinx3`, `mcf×2`, `omnetpp`, . . .); 3M + 7M instr.

Metric: IPC ↑ vs LRU. **Constraint:** ≤256 B total replacement-policy metadata across the whole LLC; code ≤1000 lines.

Table 22: The final suite covers 20 concrete architecture-design challenges. Each row lists the deliverable, metric, baseline, and constraint used by the verifier.

Family	Subfield	Simulator	Metric	Deliverable	Baseline	Constraint	Source
branch_predictor	CPU core	ChampSim	MPKI ↓	C++ direction predictor [29]	reference	16 KB storage	community
btb	CPU core	ChampSim	MPKI ↓	C++ branch target buffer	reference	N/A	community
cache_replacement	CPU core	ChampSim	IPC ↑	C++ LLC replacement policy	LRU	256 B metadata	community
l1d_prefetcher	CPU core	ChampSim	IPC ↑	C++ L1D prefetcher	reference	N/A	community
compose_bp_btb	CPU core	ChampSim	MPKI ↓	branch predictor + BTB co-design	reference	joint 32 KB	community
compose_replacement_prefetcher	CPU core	ChampSim	IPC ↑	LLC replacement + prefetcher co-design	reference	N/A	community
gem5_cache	system	gem5	IPC ↑	cache hierarchy + prefetcher config	reference	N/A	community
dramsys_ddr4	memory	DRAMSys	GB/s ↑	DDR4 controller policy tuning	reference	N/A	community
ramulator_rowhammer	memory	Ramulator 2.0	overhead ↓	RowHammer mitigation tuning [20]	reference	fixed protection level	community
astrasim_collective	system	ASTRA-sim	cycles ↓	All-Reduce algorithm + config [42]	reference	8 NPUs, fixed topology	community
astrasim_allgather	system	ASTRA-sim	cycles ↓	All-Gather algorithm + config [42]	reference	8 NPUs, fixed topology	community
astrasim_alltoall	system	ASTRA-sim	cycles ↓	All-to-All algorithm + config [42]	reference	8 NPUs, fixed topology	community
astrasim_alltoall_16	system	ASTRA-sim	cycles ↓	All-to-All algorithm + config [42]	reference	16 NPUs, fixed topology	community
astrasim_reducescatter	system	ASTRA-sim	cycles ↓	Reduce-Scatter algorithm + config [42]	reference	8 NPUs, fixed topology	community
scalesim_mc	accelerator	SCALE-Sim v3	cycles ↓	multi-core partition schedule	reference	ViT-Base [8], 128×128 array	community
scalesim_256array	accelerator	SCALE-Sim v3	cycles ↓	multi-core partition schedule	reference	ViT-Base [8], 256×256 array	community
scalesim_64array	accelerator	SCALE-Sim v3	cycles ↓	multi-core partition schedule	reference	ViT-Base [8], 64×64 array	community
timeloop_dosa	accelerator	Timeloop+Acc.	EDP ↓	per-layer mappings [16]	reference	ResNet-50 [15]	community
mnsim_pim	CIM	MNSIM 2.0	EDP ↓	ReRAM CIM design-space search	reference	area cap	community
gibbon_codesign	CIM	MNSIM 2.0	EDP ↓	NN + CIM co-design	Gibbon	CIFAR-10 [25] floor 0.738, area cap	published

Deliverables: candidate.{h,cc}; workload/principles/evaluation docs; prediction.json.

Verifier-attempt cap (L1/L2/L3): 10 / 1 / 1.

4. L1D Prefetcher l1d_prefetcher (CPU core, ChampSim)

Objective: Design the candidate L1D prefetcher maximizing average IPC on memory-intensive traces.

Workload: 3 memory-intensive SPEC traces (mcf, libquantum, mcf); 3M + 7M instr.

Metric: IPC ↑ vs next-line L1D prefetcher. **Constraint:** no storage cap; code ≤ 1500 lines.

Deliverables: candidate.{h,cc}; three design docs; pytest surrogate; prediction.json.

Verifier-attempt cap (L1/L2/L3): 10 / 1 / 1.

5. Front-End Co-Design: BP + BTB compose_bp_btb (CPU core, ChampSim)

Objective: Co-design a branch predictor and a BTB that jointly minimize average MPKI under a shared storage budget.

Workload: 3 SPEC CPU2017 traces (perlbench, gcc, xalancbmk); 3M + 7M instr.

Metric: MPKI ↓ vs reference BP+BTB. **Constraint:** joint ≤ 32 KB storage; code ≤ 2500 lines.

Deliverables: candidate_bp.{h,cc}, candidate_btb.{h,cc}; three docs; pytest; prediction.json.

Verifier-attempt cap (L1/L2/L3): 10 / 1 / 1.

6. LLC Co-Design: Replacement + Prefetcher

compose_replacement_prefetcher (CPU core, ChampSim)

Objective: Co-design the LLC replacement policy and LLC prefetcher under a single shared storage budget to maximize IPC.

Workload: 3 SPEC CPU2017 traces (mcf×2, omnetpp); 3M + 7M instr.

Metric: IPC ↑ vs reference repl.+pf. **Constraint:** shared ≤ 64 KB storage; code ≤ 2000 lines.

Deliverables: candidate_repl.{h,cc}, candidate_llc_pf.{h,cc}; three docs; pytest; prediction.json.

Verifier-attempt cap (L1/L2/L3): 10 / 1 / 1.

7. Cache Hierarchy + Prefetcher gem5_cache (System, gem5)

Objective: Design the full on-die cache subsystem (L1I/L1D, optional L2/LLC) plus prefetcher for a single-CPU SE-mode system as a config.py.

Workload: Single-CPU gem5 SE-mode workload.

Metric: IPC ↑ vs reference config. **Constraint:** on-die cache area budget; code ≤ 400 lines.

Deliverables: config.py; workload/principles/evaluation docs.

Verifier-attempt cap (L1/L2/L3): 5 / 1 / 1.

8. DDR4 Memory Controller Tuning dramsys_ddr4 (Memory, DRAMSys)

Objective: Design a DDR4 controller configuration maximizing sustained read+write bandwidth, subject to a latency deadline.

Workload: Fixed DDR4 access trace.

Metric: GB/s ↑ vs reference config. **Constraint:** latency deadline; code ≤ 200 lines.

Deliverables: config.json, mc_config.json; three design docs.

Verifier-attempt cap (L1/L2/L3): 5 / 5 / 1.

9. RowHammer Mitigation Overhead ramulator_rowhammer (Memory, Ramulator 2.0)

Objective: Pick and tune a RowHammer mitigation [20] for least performance overhead while staying provably secure at a fixed threshold.

Workload: Fixed memory access trace.

Metric: overhead % ↓ vs reference mitigation. **Constraint:** provably secure at $T_{RH}=100$; code ≤ 120 lines.

Deliverables: config.yaml (single mitigation entry); three design docs.

Verifier-attempt cap (L1/L2/L3): 5 / 5 / 1.

10. Collective: All-Reduce astrasim_collective (System, ASTRA-sim)

Objective: Configure ASTRA-sim’s System layer (collective algorithm + parameters) to minimize completion time on fixed hardware, following collective-communication optimization practice [42].

Workload: All-Reduce, 8 NPUs, fixed topology + workload.

Metric: cycles ↓ vs reference algorithm+config. **Constraint:** 8 NPUs, fixed topology; code ≤ 60 lines.

Deliverables: system.json; workload/principles/evaluation docs.
Verifier-attempt cap (L1/L2/L3): 5 / 5 / 1.

11. Collective: All-Gather astrasim_allgather (*System, ASTRA-sim*)

Objective: Configure the System layer to minimize All-Gather completion time on fixed hardware [42].

Workload: All-Gather, 8 NPUs, fixed topology + workload.

Metric: cycles ↓ vs reference algorithm+config. **Constraint:** 8 NPUs, fixed topology; code ≤ 60 lines.

Deliverables: system.json; three design docs.

Verifier-attempt cap (L1/L2/L3): 5 / 5 / 1.

12. Collective: All-to-All (8 NPUs) astrasim_alltoall (*System, ASTRA-sim*)

Objective: Configure the System layer to minimize All-to-All completion time on fixed hardware [42].

Workload: All-to-All, 8 NPUs, fixed topology + workload.

Metric: cycles ↓ vs reference algorithm+config. **Constraint:** 8 NPUs, fixed topology; code ≤ 60 lines.

Deliverables: system.json; three design docs.

Verifier-attempt cap (L1/L2/L3): 5 / 5 / 1.

13. Collective: All-to-All (16 NPUs) astrasim_alltoall_16 (*System, ASTRA-sim*)

Objective: Configure the System layer to minimize All-to-All completion time at larger scale [42].

Workload: All-to-All, 16 NPUs, fixed topology + workload.

Metric: cycles ↓ vs reference algorithm+config. **Constraint:** 16 NPUs, fixed topology; code ≤ 60 lines.

Deliverables: system.json; three design docs.

Verifier-attempt cap (L1/L2/L3): 5 / 5 / 1.

14. Collective: Reduce-Scatter astrasim_reducescatter (*System, ASTRA-sim*)

Objective: Configure the System layer to minimize Reduce-Scatter completion time on fixed hardware [42].

Workload: Reduce-Scatter, 8 NPUs, fixed topology + workload.

Metric: cycles ↓ vs reference algorithm+config. **Constraint:** 8 NPUs, fixed topology; code ≤ 60 lines.

Deliverables: system.json; three design docs.

Verifier-attempt cap (L1/L2/L3): 5 / 5 / 1.

15. Multi-Core Partitioning (128×128) scalesim_mc (*Accelerator, SCALE-Sim v3*)

Objective: Design a per-layer partitioning policy for a 4-core systolic-array accelerator running ViT-Base [8].

Workload: ViT-Base [8] (14 layers); 4 cores, each 128×128 PE array, 256 KB SRAMs, BW=10.

Metric: cycles ↓ vs reference schedule. **Constraint:** fixed 4×128×128 hardware; code ≤ 2000 lines.

Deliverables: config.cfg; workload/principles/evaluation docs.

Verifier-attempt cap (L1/L2/L3): 5 / 5 / 1.

16. Multi-Core Partitioning (256×256) scalesim_256array (*Accelerator, SCALE-Sim v3*)

Objective: Design a per-layer partitioning policy for a 4-core accelerator with larger arrays.

Workload: ViT-Base [8] (14 layers); 4 cores, each 256×256 PE array, 256 KB SRAMs, BW=10.

Metric: cycles ↓ vs reference schedule. **Constraint:** fixed 4×256×256 hardware; code ≤ 2000 lines.

Deliverables: config.cfg; three design docs.

Verifier-attempt cap (L1/L2/L3): 5 / 5 / 1.

17. Multi-Core Partitioning (64×64) scalesim_64array (*Accelerator, SCALE-Sim v3*)

Objective: Design a per-layer partitioning policy for a 4-core accelerator with smaller arrays.

Workload: ViT-Base [8] (14 layers); 4 cores, each 64×64 PE array, 256 KB SRAMs, BW=10.

Metric: cycles ↓ vs reference schedule. **Constraint:** fixed 4×64×64 hardware; code ≤ 2000 lines.

Deliverables: config.cfg; three design docs.

Verifier-attempt cap (L1/L2/L3): 5 / 5 / 1.

18. Per-Layer Mapping (DOSA [16]) timeloop_dosa (*Accelerator, Timeloop+Accelergy*)

Objective: Choose the Timeloop mapping (loop order, tiling, spatial parallelism) for every ResNet-50 layer [15], minimizing geomean EDP.

Workload: ResNet-50 [15] (24 unique layers) on one fixed DNN accelerator.

Metric: geomean EDP ↓ vs reference mapping. **Constraint:** fixed Gemmini-style array; code ≤ 30000 lines.

Deliverables: 24 per-layer mapping *.yaml; three design docs.

Verifier-attempt cap (L1/L2/L3): 10 / 5 / 1.

19. ReRAM CIM Design Search mnsim_pim (*CIM, MNSIM 2.0*)

Objective: Search MNSIM's device / crossbar / interface knobs for one ReRAM CIM accelerator (SimConfig.ini) minimizing EDP under an area cap.

Workload: MNSIM ReRAM CIM design point.

Metric: EDP ↓ vs reference design. **Constraint:** area cap; code ≤ 2000 lines.

Deliverables: SimConfig.ini; design_analysis, principles, evaluation docs.

Verifier-attempt cap (L1/L2/L3): 5 / 1 / 1.

20. NN + CIM Co-Design (Gibbon) gibbon_codesign (*CIM, MNSIM 2.0*)

Objective: Co-design the network and the ReRAM-CIM accelerator (design.json) minimizing EDP while meeting an accuracy floor and area cap.

Workload: CIFAR-10 [25] network + ReRAM-CIM accelerator.

Metric: EDP ↓ vs Gibbon (published). **Constraint:** CIFAR-10 [25] accuracy floor 0.738, area cap; code ≤ 2000 lines.

Deliverables: design.json; design_analysis, principles, evaluation docs.

Verifier-attempt cap (L1/L2/L3): 5 / 3 / 1.

B Detailed Cross-Agent Results

This appendix expands the body's cross-agent comparison (Table 10) using the final single-seed suite. Table 23 reports cost and runtime by setting. Tables 24, 25, and 26 break down L3 performance prediction: whether the agent produced enough measurable artifacts, how accurate its submitted-design self-evaluation was, and how well agent-written models ranked candidate designs. Table 27 gives the per-challenge L3 baseline-normalized performance values behind the suite aggregates.

C Representative Agent Trajectory Excerpts

To make the evaluation settings concrete, this appendix shows one representative trajectory per agent configuration and setting on a single shared challenge, branch_predictor: design a C++ branch direction predictor, under a 16 KB metadata budget and no dynamic allocation, that minimizes average MPKI across three SPEC CPU2017 traces (perlbench, gcc, xalancbmk) versus ChampSim's stock bimodal. These excerpts are illustrative; Section 5 reports aggregate behavior across the full suite. For each run we show

Table 23: Evaluation support changes both agent effort and runtime. Entries report mean tokens, turns, and minutes per agent-challenge run. GPT-5.5 minutes use observed simulator time because agent wall time is unavailable.

Agent		L1 (full harness)			L2 (simulator-code container)			L3 (agent-only)		
Model	Framework	Mtok↓	turns↓	min↓	Mtok↓	turns↓	min↓	Mtok↓	turns↓	min↓
GPT-5.5	Codex	3.93	n/a	14.8	2.09	n/a	3.4	1.43	n/a	2.5
Gemini 3.5 Flash	MiniSWE	2.83	75.4	18.1	3.86	101.3	15.6	2.67	101.0	7.1
Gemini 3.1 Flash-Lite	MiniSWE	0.16	20.9	8.9	0.25	23.1	3.3	0.19	22.8	1.0
Gemma 4 31B	MiniSWE	0.31	25.2	17.0	0.74	43.8	14.0	0.59	41.3	15.8

Table 24: Most L3 sessions do not produce a measurable agent-written model comparison. Counts drop when agents save too few candidate designs, do not provide an executable or design-sensitive performance model, or have too few candidates that run in the verifier simulator.

Agent		3+ designs ↑	executable model ↑	model varies ↑	3+ valid runs ↑	agreement measured ↑
Model	Framework					
GPT-5.5	Codex	19	18	15	9	9
Gemma 4 31B	MiniSWE	14	13	7	7	7
Gemini 3.5 Flash	MiniSWE	4	3	0	0	0
Gemini 3.1 Flash-Lite	MiniSWE	5	2	0	0	0

Table 25: Even executable L3 performance models weakly match the verifier simulator. Columns use the same submitted-design self-evaluation definitions as Table 13; the funnel in Table 24 explains which runs have enough saved candidates for within-session agreement.

Agent		exec. model ↑	agreement ↑	self-eval given ↑	median error ↓	range hit ↑
Model	Framework	%	τ	%		
GPT-5.5	Codex	90%	+0.20	100%	93%	15%
Gemini 3.5 Flash	MiniSWE	15%	n/a	45%	81%	22%
Gemini 3.1 Flash-Lite	MiniSWE	10%	n/a	85%	96%	0%
Gemma 4 31B	MiniSWE	65%	+0.13	80%	99%	12%

the visible rationale and action trace available in the log: a model-provided rationale trace when exposed by the agent framework, or the visible transcript and tool/action trace otherwise. [...] marks an elision, and [bracketed notes] stand in for elided actions (file writes, builds, simulator runs).

The agent configurations differ in what rationale fields the framework exposes. GPT-5.5 and Gemma 4 31B expose model-provided rationale traces in every setting. The Gemini runs do not expose a separate rationale field. For those runs, we show the visible transcript when it contains rationale; otherwise we show the action trace together with rationale embedded in shell comments and in the surrogate code the agent wrote.

Across the trajectories, three behaviors recur, and the evaluation setting controls how each is done: *workload analysis* (what the traces look like and where bimodal loses), *design* (the predictor chosen under the 16 KB budget), and *performance prediction* (how a design is judged before the verifier submission: the real ChampSim oracle at L1, the real build at L2, and an agent-written Python surrogate over static workload evidence at L3).

C.1 GPT-5.5

GPT-5.5 compresses a whole session into one upfront rationale block, then commits in very few steps. The workload analysis, design, and performance-prediction plan are all visible in that single block.

L1 (full harness). Probes the baseline through the real oracle, then designs a budgeted hybrid.

```

I'll inspect the starter, trace format, and validation limits
↪ first, then build a
compact predictor and iterate with the ChampSim oracle while
↪ logging each result.
[...]
The simulator has the real ChampSim traces and a decoded directory
↪ inside its
container. I'm going to sample those decoded files to identify
↪ which three workloads
are used and what branch patterns are visible.
The root implementation is still the 1 KB starter and passes
↪ storage. I'm submitting
it as a baseline probe now so the oracle reveals the actual three
↪ measured traces and
baseline MPKI shape.

```

Table 26: Only 16 of 80 L3 runs have enough evidence to compare the agent’s performance model with the verifier simulator. We list only runs with computable Kendall τ ; all omitted runs fail an earlier gate in Table 24.

Agent	Task	Domain	Kendall $\tau \uparrow$	Reading
GPT-5.5 + Codex	L1 data prefetcher	CPU core	+1.00	correct ordering
GPT-5.5 + Codex	Branch predictor and BTB co-design	CPU core	+0.80	mostly correct ordering
GPT-5.5 + Codex	LLC replacement and prefetcher co-design	CPU core	+0.20	weak ordering
GPT-5.5 + Codex	Cache hierarchy configuration	system	−1.00	reversed ordering
GPT-5.5 + Codex	RowHammer mitigation	memory	+0.10	near-random ordering
GPT-5.5 + Codex	ReduceScatter collective communication	system	−0.40	wrong ordering
GPT-5.5 + Codex	Multi-core accelerator mapping	accelerator	+0.60	useful but imperfect ordering
GPT-5.5 + Codex	256-array accelerator mapping	accelerator	−0.40	wrong ordering
GPT-5.5 + Codex	CIM accelerator co-design	CIM	+0.20	weak ordering
MiniSWE + Gemma 4 31B	AllReduce collective communication	system	+0.13	weak ordering
MiniSWE + Gemma 4 31B	AllGather collective communication	system	−0.12	near-random ordering
MiniSWE + Gemma 4 31B	AllToAll collective communication	system	+0.76	mostly correct ordering
MiniSWE + Gemma 4 31B	16-NPU AllToAll collective communication	system	+0.20	weak ordering
MiniSWE + Gemma 4 31B	ReduceScatter collective communication	system	+0.11	near-random ordering
MiniSWE + Gemma 4 31B	Multi-core accelerator mapping	accelerator	+0.20	weak ordering
MiniSWE + Gemma 4 31B	256-array accelerator mapping	accelerator	−0.80	strongly reversed ordering

Computable runs / median τ : GPT-5.5 + Codex has 9 runs with median +0.20; MiniSWE + Gemma 4 31B has 7 runs with median +0.13.

Runs with no computable τ : MiniSWE + Gemini 3.5 Flash has 20/20; MiniSWE + Gemini 3.1 Flash-Lite has 20/20.

Table 27: L3 performance varies sharply by challenge and agent. Values report baseline-normalized performance; 1.0× matches the challenge baseline and larger is better. *hf* marks no valid verifier submission. Bottom rows repeat the suite aggregates.

Family	Subfield	Relative performance $\uparrow$			
		GPT-5.5 + Codex	MiniSWE + Gem. 3.5 Flash	MiniSWE + Gem. 3.1 Flash-Lite	MiniSWE + Gemma 4
branch_predictor	CPU core	1.03	0.68	1.03	0.99
btb	CPU core	1.26	0.96	1.02	0.99
cache_replacement	CPU core	0.91	0.97	0.94	0.95
l1d_prefetcher	CPU core	1.04	0.74	0.81	0.81
compose_bp_btb	CPU core	0.99	0.99	0.66	0.85
compose_replacement _prefetcher	CPU core	0.93	0.92	1.00	<i>hf</i>
gem5_cache	system	1.88	1.63	<i>hf</i>	1.69
drmsys_ddr4	memory	0.87	0.87	0.88	<i>hf</i>
ramulator_rowhammer	memory	1.42	5.52	1.59	1.42
astrasim_collective	system	3.29	0.16	0.16	0.16
astrasim_allgather	system	0.15	0.15	0.15	1.79
astrasim_alltoall	system	1.00	0.05	0.05	0.05
astrasim_alltoall_16	system	1.30	0.03	0.02	0.08
astrasim_reducescatter	system	0.65	0.16	0.16	0.16
scalesim_mc	accelerator	1.34	1.34	1.33	1.33
scalesim_256array	accelerator	0.80	0.73	0.80	0.80
scalesim_64array	accelerator	2.30	2.30	2.01	2.29
timeloop_dosa	accelerator	3.28	3.30	0.15	2.57
mnsim_pim	CIM	1.92	<i>hf</i>	<i>hf</i>	<i>hf</i>
gibbon_codesign	CIM	2.62	<i>hf</i>	<i>hf</i>	1.80
Suite geomean $\uparrow$		1.21	0.61	0.45	0.72
Suite median $\uparrow$		1.15	0.90	0.81	0.99
Beat baseline $\uparrow$		65%	25%	30%	35%

The synchronous oracle call hit the 120 second tool timeout, so I'm
 $\hookrightarrow$ switching to
 async submission and polling.
 While the baseline run is in flight, I'm preparing a conservative
 $\hookrightarrow$ design: a hashed
 gshare-style table plus a small chooser and bimodal fallback, all
 $\hookrightarrow$ as fixed bitfield
 arrays under 16 KB. It should recover branches whose direction
 $\hookrightarrow$ depends on recent
 outcomes without giving up simple per-PC behavior.

L2 (simulator-code container). Runs offline predictors over the trace pool and guards against overfitting the hidden judge trio.

I'll inspect the provided predictor interface, trace format, and
 $\hookrightarrow$ existing ChampSim
 predictors first, then build a small trace-analysis loop so the
 $\hookrightarrow$ design is tied to
 the three actual workloads.
 [...]
 I found the decoded trace pool under
 $\hookrightarrow$ /work/workload_pools/champsim/decoded. Next I'm
 checking the harness/baseline metadata to identify the exact three
 $\hookrightarrow$ traces used by

the judge, then I'll run fast offline predictors over those decoded
 ↪ files before
 committing to hardware.
 There is no visible baseline.json or evaluate.sh, so the exact
 ↪ judge trio is not
 directly exposed. I'll design against the SPEC2017 decoded traces
 ↪ in the pool and
 then run the simulator across likely candidates where practical,
 ↪ aiming for a
 predictor that improves broadly over bimodal rather than
 ↪ overfitting one file.

L3 (agent-only). With no runnable simulator feedback, it writes its own surrogate and lets the provided workload evidence pick the design.

I'll inspect the workspace, starter predictor, trace samples, and
 ↪ validation checks
 first, then build a surrogate so the C++ design choices are driven
 ↪ by the visible
 branch behavior.
 The starter is just a 4K-entry 2-bit bimodal. [...]
 The task package includes decoded branch-record samples but no
 ↪ runnable ChampSim
 workflow. I'm going to use only enough branch records for surrogate
 ↪ iteration,
 keeping the test code independent of simulator execution.
 For the actual predictor, I'm going with a compact tagged
 ↪ global-history design
 because the visible traces show many repeated static branches and
 ↪ the stock
 bimodal's main weakness is loss of context.

C.2 Gemini 3.5 Flash

Gemini 3.5 Flash did not expose a separate rationale field; on L1 and L3 branch-predictor runs we show the action trace, while on L2 the visible transcript included extended rationale.

L1 (full harness). No separate rationale trace; ran a real-submit loop on the generous L1 budget.

```
[no separate rationale trace emitted on this run; action trace]
list_files -> read starter candidate.{h,cc} -> validate.py
↪ (--budget 16384)
-> sample /traces/decoded/600.perlbench_s ... (head)
-> file_change (write candidate) -> submit_and_wait
-> browse_simulator /work/runtimes/champsim/{branch,inc,src}
-> read_simulator_file inc/modules.h
[then ~10 rounds of: file_change -> validate.py -> submit_and_wait]
-> session_end ("We have successfully designed ...")
```

L2 (simulator-code container). The clearest "build the evaluation loop" struggle: a broken checker, then a stale-build bug, then a TAGE design.

```
[validate.py] failed because of `libclang.so: cannot open shared
↪ object file: No
such file or directory`. [...] `apt-get install -y libclang-dev`
↪ failed because of
permission denied (we are user `agent`, not root).
[...]
Wait, why did gshare have the exact same results as bimodal? [...]
all the branch predictors are linked into the binary! But how does
↪ it select which
one to run? [...]
Wait! In the last make run, it only compiled src/main.cc and linked
↪ the binary! It
did NOT recompile src/generated_environment.cc!
[fix: rm .csconfig/generated_environment.o && make; now
↪ bimodal/gshare/hashed_
perceptron/candidate give distinct MPKI on the 3 traces]
[...]
To proceed, we need to write our implementation of
↪ candidate.{h,cc}. We will write a
```

fully compliant, self-contained TAGE predictor with 6 tagged tables
 ↪ and 1 base table.

L3 (agent-only). No separate rationale trace; uses the provided interface and submits close to the starter.

```
[no separate rationale trace emitted on this run; action trace]
list_files -> read starter -> sample trace (head) -> validate.py
-> cp starter candidate.* -> submit_and_wait
```

C.3 Gemini 3.1 Flash-Lite

Gemini 3.1 Flash-Lite does not expose a separate rationale field. Its rationale is visible only in the comments of the shell commands it runs and in the surrogate code it writes, which we show here.

L1 (full harness). Attempts a workload measurement, then guesses the baseline instead of measuring it.

```
[no separate rationale trace; rationale appears only in shell
↪ comments]
# Let's count total branches and taken branches in a small sample
awk '{if ($3 == 1) {total++; if ($4 == 1) taken++;}} END {print
↪ "Total:",total,...}'
[...]
# Need to find the baseline MPKI to compare against.
# The task says "versus ChampSim's stock bimodal".
# I should have run the baseline first, but I didn't.
# Let me try to find the baseline MPKI. I can't run the baseline
↪ directly,
# but I can assume the average MPKI is around 8-10.
[writes candidate + the docs] -> session_end
```

L2 (simulator-code container). Designs a gshare with hand-checked budget, cannot get the real sim to finish, and ships estimated numbers.

```
[no separate rationale trace; rationale in shell comments + the
↪ code it writes]
# 16384 bytes = 131072 bits. If we have 2-bit counters, 65536
↪ counters. 16-bit GHR.
# Let's write a GShare implementation
[writes candidate.{h,cc}: GShare, GHR_BITS=14, table 2^14, 2-bit
↪ counters]
# Manual check of memory usage: 16384 * 2 = 32768 bits = 4096 bytes
↪ ... within budget
[builds ChampSim, debugs --simulation-instructions usage, run
↪ deadlocks]
# The deadlock might be due to the GShare implementation being too
↪ simple [...]
[writes evaluation.md with an ESTIMATED per-trace MPKI table]
*Note: These are estimates based on typical GShare performance
↪ improvements.*
```

L3 (agent-only). No separate rationale trace, but it still builds a real Python surrogate and sweeps it against a measured bimodal baseline, with the rationale encoded as code.

```
[no separate rationale trace; the rationale is in the surrogate
↪ code it writes and runs]
[writes bimodal_surrogate.py: 4K-entry bimodal, computes MPKI over
↪ the 3 decoded
traces -> establishes the baseline]
[writes gshare_surrogate.py, then gshare_v2..v6.py: sweeps
↪ table_size in {4K,8K} and
history_length in {10,12,13,14}, index = ((ip>2) ^ history) %
↪ table_size]
[writes tests/test_predictor.py: asserts avg_mпки < 62.55 # the
↪ measured bimodal]
-> submit_and_wait
```

C.4 Gemma 4 31B

Gemma 4 31B exposes many small rationale steps in every setting, with explicit budget arithmetic throughout. Its performance-prediction path changes with the setting: the real oracle at L1, the

(broken) real build at L2, and a Python surrogate over static workload evidence at L3.

does not.]

L1 (full harness). Plans workload-then-design, sizes the table to the budget, and reads back per-trace MPKI from real submissions.

The goal is to design a branch direction predictor to minimize
 ↳ average MPKI across
 three SPEC CPU2017 traces. [...] Constraint: Metadata <= 16384
 ↳ bytes. No dynamic
 allocation.
 Plan: 1. Analyze the traces to understand the branch patterns. 2.
 ↳ Implement a
 predictor (e.g., Gshare, TAGE, or a hybrid) that fits within the
 ↳ budget.
 3. Validate the storage budget. 4. Submit and evaluate. 5. Iterate
 ↳ based on results.
 [...]
 Budget check for Gshare: Table: 2^{14} entries * 2 bits = 4096 bytes.
 ↳ [...]
 2^{16} entries * 2 bits = 16384 bytes. (Exactly the budget).
 [...]
 The first submission (Gshare with 15-bit GHR) yielded an average
 ↳ MPKI of 6.37.
 Per-trace MPKI: 600.perlbench 2.523, 602.gcc 9.097, 623.xalancbmk
 ↳ 7.482.
 [...] I am over [budget] by 32 bits. [...] let's try a TAGE-like or
 ↳ a Tournament
 predictor (combining bimodal and gshare).

L2 (simulator-code container). Engages the real interface (SFINAE, not virtual), but the build deadlocks, so it falls back to an estimated reduction.

the branch_predictor base class in modules.h does NOT have virtual
 ↳ functions. It uses
 a template-based "has_member" check (SFINAE) to determine if the
 ↳ derived class
 implements certain methods. This is why `override` failed. I need
 ↳ to remove override.
 [...]
 Stock bimodal uses a table of 16384 entries [...] = 4096 bytes.
 ↳ [...]
 I'm having trouble running the simulator due to deadlocks. I'll
 ↳ focus on a solid
 gshare-like implementation and then submit.
 [writes evaluation.md:] Predicted MPKI reduction: 15-30% compared
 ↳ to bimodal.
 Validation was attempted using the ChampSim simulator, but
 ↳ simulation was interrupted
 by deadlocks [...] The design follows the standard Gshare
 ↳ architecture known to
 outperform bimodal.

L3 (agent-only). Textbook performance prediction: a Python surrogate measures four designs and the budget-feasible best is chosen.

Plan: 1. Create a Python surrogate [...] 3. Experiment with
 ↳ different predictor
 designs (Gshare, TAGE-like) using the surrogate. 4. Select the best
 ↳ design.
 [...]
 Gshare (12-bit history, 4096 entries) is already better than
 ↳ Bimodal on average.
 Bimodal: 52.50 MPKI; Gshare: 44.44 MPKI. [...] Gshare is worse on
 ↳ perlbench and
 xalancbmk, but much better on gcc.
 [...] 2^{16} entries * 2 bits = 16384 bytes. This fits exactly into
 ↳ the budget. [...]
 For hb=16, Avg MPKI was 32.77. Bimodal was 52.50. This is a
 ↳ significant improvement.
 [...] The current design is a Tournament Predictor with 16K entries
 ↳ for Bimodal,
 Gshare, and Choice tables. Average MPKI (surrogate): ~28.68.
 [...] Tournament 16K * 2 bits * 3 tables = 96K bits = 12KB. This
 ↳ fits. [32K = 24KB]